



Informe técnico — PROYECTO WEB

Proyecto WEB 2026

Universidad del Magdalena

Abril 2026

Versión: v0.1 (borrador técnico)



ÍNDICE

0. Cómo usar este documento	6
0.1. Qué se espera que aprenda un compañero al leerlo	6
0.2. Ruta rápida de lectura (si solo tiene 20–30 minutos)	7
0.3. Ruta completa de lectura (si quiere entender todo el proyecto)	8
0.4. Glosario básico de términos	9
1. Contexto del problema y propósito del proyecto	12
1.1. Problema real que busca resolver la API	12
1.2. Por qué este problema importa en un contexto universitario	13
1.3. Qué evita el sistema (duplicidad, traslapes, mala trazabilidad, etc.)	13
1.4. Qué logra el sistema al final	14
2. Qué construyó exactamente el equipo	15
2.1. Tipo de sistema entregado	15
2.2. Qué módulos funcionales incluye	16
2.3. Qué usuarios o roles implícitos interactúan con el sistema	17
2.4. Qué cosas sí cubre el proyecto y qué cosas no cubre	18
3. Tecnologías usadas y qué aporta cada una	19
3.1. Java 21	19
3.2. Spring Boot 4	20
3.3. PostgreSQL	20
3.4. Spring Data JPA	21
3.5. Testcontainers	21
3.6. JUnit 5	22
3.7. Mockito	22
3.8. Lombok	23
3.9. Mermaid documentación y requests.http	23
4. Arquitectura general del proyecto	24
4.1. Qué significa arquitectura por capas	24
4.2. Flujo general de una petición dentro del proyecto	25
4.3. Cómo se relacionan controller service repository y base de datos	26
4.4. Por qué esta arquitectura ayuda a organizar el proyecto	26
5. Estructura de carpetas del proyecto	27



5.1. Vista general del árbol principal.....	28
5.2. entity/	28
5.3. repository/	29
5.4. dto/	29
5.5. service/.....	30
5.6. service/impl/	30
5.7. controller/	30
5.8. mapper/	31
5.9. exception/.....	31
5.10. src/test/java/	32
5.11. Archivos raíz importantes: README.md, MER.md, requests.http, pom.xml	32
6. Archivos más importantes por carpeta y qué hace cada uno	32
6.1. Archivos clave de entity/	33
6.2. Archivos clave de repository/	33
6.3. Archivos clave de service/impl/.....	34
6.4. Archivos clave de controller/.....	35
6.5. Archivos clave de exception/	35
6.6. Archivos clave de src/test/	36
7. Modelo de datos explicado en lenguaje sencillo	36
7.1. Las 7 entidades obligatorias	37
7.2. Qué representa cada entidad	38
7.3. Relaciones entre entidades	39
7.4. Campos más importantes de cada entidad.....	40
7.5. Estados del sistema `AppointmentStatus`, `PatientStatus`, `OfficeStatus`	41
8. Módulos funcionales del sistema	42
8.1. Pacientes	42
8.2. Doctores.....	43
8.3. Especialidades	43
8.4. Consultorios	44
8.5. Tipos de cita.....	44
8.6. Horarios de doctores	44
8.7. Citas médicas.....	45



8.8. Disponibilidad	45
8.9. Reportes.....	46
9. Endpoints principales y qué hace cada uno	46
9.1. Tabla resumen de endpoints por módulo.....	47
9.2. Endpoints más importantes que sí o sí deben saberse.....	51
9.3. Ejemplos sencillos de uso	52
9.4. Códigos HTTP más comunes del proyecto.....	56
10. Reglas de negocio más importantes del proyecto	57
10.1. Reglas al crear una cita.....	57
10.2. Reglas para confirmar una cita	58
10.3. Reglas para cancelar una cita	59
10.4. Reglas para completar una cita	59
10.5. Reglas para marcar no-show.....	59
10.6. Reglas de disponibilidad.....	60
10.7. Reglas de reportes	60
11. El corazón del proyecto: flujo completo de una cita.....	61
11.1. Qué pasa cuando se crea una cita	61
11.2. Cómo se valida si hay conflicto	61
11.3. Cómo se calcula endAt.....	62
11.4. Cómo cambia el estado de una cita.....	62
11.5. Qué pasa si se cancela	63
11.6. Qué pasa si se completa	63
11.7. Qué pasa si se marca como no asistida	63
12. Cómo se calcula la disponibilidad	64
12.1. Qué información se usa.....	64
12.2. Cómo se generan los slots	64
12.3. Qué hace que un slot esté libre u ocupado.....	65
12.4. Ejemplo práctico paso a paso.....	65
13. Cómo funcionan los reportes	66
13.1. Reporte de ocupación de consultorios.....	66
13.2. Reporte de productividad de doctores	66
13.3. Reporte de pacientes con más no-show.....	67



13.4. Qué consultas del repository soportan estos reportes	67
13.5. Qué significan los datos agregados.....	67
14. Qué se prueba y por qué se prueba.....	68
14.1. Qué son pruebas unitarias.....	68
14.2. Qué son pruebas de integración.....	68
14.3. Qué son pruebas de controller	69
14.4. Qué archivos de test existen en este proyecto	69
14.5. Qué protege cada grupo de pruebas	70
14.6. Qué significa que mvn clean test pase con todos los tests	70
15. Archivos que sí o sí deben conocerse para defender el proyecto	70
15.1. Top 10 archivos más importantes.....	71
15.2. Qué responder si preguntan por cada uno	72
15.3. Qué no vale la pena memorizar en detalle	72
16. Preguntas probables de sustentación y cómo responderlas	73
16.1. Preguntas de arquitectura	73
16.2. Preguntas de reglas de negocio	74
16.3. Preguntas de base de datos.....	74
16.4. Preguntas de reportes	75
16.5. Preguntas de testing.....	75
16.6. Preguntas sobre decisiones técnicas	76
17. Errores comunes o confusiones que deben evitar los compañeros.....	76
17.1. No confundir entidad con DTO	77
17.2. No confundir repository con service.....	77
17.3. No decir que el controller “hace toda la lógica”	77
17.4. No olvidar que endAt lo calcula el servidor	77
17.5. No olvidar los 3 tipos de reportes principales.....	77
17.6. No olvidar las transiciones válidas de estado	77
18. Resumen final del proyecto en una página	78
18.1. Qué problema resuelve	78
18.2. Cómo está construido.....	78
18.3. Qué reglas importantes cumple	78
18.4. Qué pruebas lo respaldan	78



18.5. Qué lo hace defendible frente a la rúbrica	78
--	----

0. Cómo usar este documento

Este documento fue creado para que cualquier integrante del grupo, incluso si no domina Java, Spring Boot o bases de datos, pueda entender el proyecto con suficiente claridad para participar en la sustentación. No está pensado únicamente como una descripción técnica del código, sino como una guía de estudio y comprensión progresiva del sistema.

La idea principal es que el lector no se pierda entre nombres de carpetas, clases o conceptos de programación, sino que primero entienda qué problema resuelve el sistema, cómo está organizado, cuál es su lógica principal y qué debe saber sí o sí para defenderlo correctamente. Por eso, el documento avanza desde lo más general hacia lo más específico: primero se explica el contexto del problema, luego la solución construida, después la arquitectura y la estructura del proyecto, y finalmente la lógica de negocio, las pruebas y las preguntas probables de la sustentación.

No es necesario leerlo todo de una sola vez. Puede usarse como material de repaso rápido, como guía de estudio más profunda o como apoyo para preparar respuestas concretas frente a preguntas del profesor. Lo más importante es que el lector comprenda que este proyecto no es “solo código”: es una solución backend que organiza citas médicas universitarias, valida reglas reales del negocio y demuestra calidad técnica a través de pruebas automatizadas.

0.1. Qué se espera que aprenda un compañero al leerlo

Al terminar de leer este documento, un compañero debería ser capaz de explicar con sus propias palabras qué problema resuelve el proyecto, por qué fue necesario construirlo y qué beneficios aporta frente a una gestión manual o desorganizada de citas médicas.

También debería poder identificar los módulos principales del sistema, por ejemplo: pacientes, doctores, especialidades, consultorios, tipos de cita, horarios, citas médicas, disponibilidad y reportes. No se espera que memorice todo el código, pero sí que entienda qué hace cada módulo, cómo se relaciona con los demás y por qué todos son importantes para que el sistema funcione.



Otro aprendizaje clave es entender la arquitectura general del proyecto. Es decir, comprender que el sistema está dividido por capas: una capa que recibe solicitudes HTTP, otra que aplica reglas de negocio, otra que consulta la base de datos y otra que transforma los datos para responder correctamente. Esta idea es importante porque en la sustentación pueden preguntar no solo “qué hace el sistema”, sino también “cómo está construido”.

Además, el lector debería quedar en capacidad de explicar el flujo más importante del proyecto: la creación y gestión de una cita médica. Eso incluye saber que una cita no se guarda de cualquier forma, sino que pasa por varias validaciones: el paciente debe existir y estar activo, el doctor debe existir y estar activo, el consultorio debe estar disponible, la cita no puede estar en el pasado, debe caer dentro del horario del doctor y no puede chocar con otras citas del doctor, del paciente o del consultorio.

Por último, el compañero debería poder responder preguntas básicas sobre pruebas, reportes y decisiones del proyecto. Por ejemplo: qué se prueba, por qué se prueba, qué tipos de reportes existen y qué parte del sistema demuestra mejor la complejidad del trabajo realizado.

0.2. Ruta rápida de lectura (si solo tiene 20–30 minutos)

Si un compañero tiene poco tiempo y necesita prepararse rápido, lo recomendable es que use este documento como una ruta corta de supervivencia. En ese caso, no debe intentar memorizar todo, sino enfocarse en entender las ideas más importantes del proyecto.

La primera parte que debe leer es el contexto del problema y el propósito del sistema. Si no entiende cuál era el problema inicial, difícilmente podrá justificar por qué el proyecto existe. Después debe leer la sección donde se explica qué construyó exactamente el equipo y cuáles son los módulos funcionales del sistema. Eso le dará una visión general de qué puede hacer la API.

Luego conviene leer la parte de modelo de datos y la explicación sencilla de las entidades. Allí debe quedarse, sobre todo, con una idea fundamental: la entidad central del sistema es la cita médica, porque conecta al paciente, al doctor, al consultorio y al tipo de cita. Entender eso ayuda muchísimo a comprender el resto del proyecto.



Después de eso, la mejor inversión de tiempo es leer las reglas de negocio más importantes y el flujo de una cita médica. En una sustentación, lo más probable es que los profesores pregunten cómo se crea una cita, cómo se evita que haya conflictos de horario, cómo se confirma, cómo se cancela o cómo se completa. Esa es la parte más crítica del proyecto.

Finalmente, si aún queda tiempo, debe revisar la sección de reportes, la sección de pruebas y la sección de preguntas probables de sustentación. Con eso ya tendría una base suficiente para no quedarse completamente perdido si le preguntan algo esencial.

En resumen, la ruta rápida ideal sería:

1. Contexto del problema.
2. Qué construyó el equipo.
3. Entidades principales y relación entre ellas.
4. Reglas de negocio de citas.
5. Flujo de creación y ciclo de vida de una cita.
6. Reportes principales.
7. Qué se prueba y por qué se prueba.
8. Preguntas probables de sustentación.

0.3. Ruta completa de lectura (si quiere entender todo el proyecto)

Si un integrante sí quiere comprender bien el proyecto y no solo “sobrevivir” en la sustentación, entonces debería leer el documento en el orden completo propuesto.

Primero debe leer esta sección inicial para entender cómo usar el material. Luego debe pasar al contexto del problema, porque el proyecto cobra sentido únicamente si se comprende qué falla en una gestión manual de citas. Después debe revisar qué construyó exactamente el equipo y cuál es el alcance real del sistema.

Una vez tenga esa visión general, debe entrar a las tecnologías y a la arquitectura general del proyecto. Allí no se trata de convertirse en experto en Spring Boot, sino de entender por qué el proyecto está dividido en capas y qué función cumple cada una. Ese paso es importante para ubicar mentalmente el código.



Luego debe leer la estructura de carpetas y los archivos más importantes por carpeta. Esta parte le ayudará a orientarse dentro del proyecto y a saber dónde buscar cada cosa. Por ejemplo, entender que las entidades están en una carpeta, los controllers en otra, los services en otra y las pruebas en otra.

Después de eso debe leer con atención el modelo de datos, los módulos funcionales, los endpoints y las reglas de negocio. Este es el corazón del documento, porque explica el comportamiento real del sistema. Allí es donde debe entenderse a fondo cómo funcionan las citas, cómo se calcula la disponibilidad y cómo se construyen los reportes.

En una lectura completa también es muy importante revisar la parte de pruebas. Muchos compañeros creen que el proyecto termina cuando “el código compila”, pero en realidad una parte importante del valor del sistema está en demostrar que fue probado. Comprender los tipos de pruebas y qué protege cada una ayuda mucho en la sustentación.

Finalmente, la lectura completa debe cerrar con la parte de preguntas probables, errores comunes, archivos que sí o sí hay que conocer y el resumen final del proyecto en una página. Estas secciones están pensadas como preparación directa para la defensa oral.

0.4. Glosario básico de términos

****API:**** Es la forma en que otros programas o clientes se comunican con el sistema. En este proyecto, la API recibe solicitudes HTTP y responde con datos en formato JSON.

****API REST:**** Es un estilo de construcción de APIs basado en rutas, métodos HTTP y recursos. Por ejemplo, una ruta como `/api/patients` representa el recurso “pacientes”.

****Backend:**** Parte del sistema que no ve directamente el usuario final. Se encarga de procesar la lógica, validar reglas, consultar la base de datos y responder solicitudes.

****Endpoint:**** Es una ruta específica de la API que sirve para realizar una acción. Por ejemplo, `POST /api/appointments` crea una cita.

****HTTP:**** Protocolo mediante el cual se envían las solicitudes entre cliente y servidor. En este proyecto se usan principalmente métodos GET, POST y PUT.



****GET:**** Método HTTP usado para consultar información. Ejemplo: listar pacientes o consultar reportes.

****POST:**** Método HTTP usado para crear nuevos registros. Ejemplo: crear un paciente o crear una cita.

****PUT:**** Método HTTP usado para actualizar información o mover una entidad a otro estado. Ejemplo: confirmar, cancelar o completar una cita.

****JSON:**** Formato de texto usado para enviar y recibir datos entre cliente y servidor. Es el formato principal de las peticiones y respuestas de la API.

****Entidad (Entity):**** Clase que representa una tabla de la base de datos. Por ejemplo, `Patient`, `Doctor` o `Appointment`.

****DTO:**** Objeto usado para transportar datos entre capas o entre cliente y servidor. Sirve para no exponer directamente las entidades de la base de datos en la API.

****Repository:**** Capa que se encarga de consultar y guardar información en la base de datos. Es la puerta de acceso a la persistencia.

****Service:**** Capa donde vive la lógica de negocio. Aquí se aplican las reglas importantes del sistema antes de consultar o guardar información.

****Controller:**** Capa que recibe las solicitudes HTTP del exterior y delega el trabajo al service correspondiente.

****Mapper:**** Clase que convierte entidades en DTOs de respuesta. Sirve para transformar datos internos del sistema a un formato adecuado para la API.

****Base de datos relacional:**** Estructura donde la información se guarda en tablas relacionadas entre sí por llaves o referencias.



Informe técnico — Proyecto Web

Versión: 1

****Foreign Key (FK):**** Campo que conecta una tabla con otra. En este proyecto, una cita guarda referencias a paciente, doctor, consultorio y tipo de cita.

****Regla de negocio:**** Condición o validación que existe porque el dominio lo necesita. Ejemplo: no permitir una cita fuera del horario del doctor.

****Transición de estado:**** Cambio de una cita de un estado a otro. Por ejemplo, de `SCHEDULED` a `CONFIRMED`.

****Enum:**** Tipo de dato que contiene una lista cerrada de valores válidos. Por ejemplo, los estados posibles de una cita.

****Excepción:**** Error controlado del sistema. Sirve para detener una operación cuando algo no cumple las reglas o no existe.

****Prueba unitaria:**** Verifica una parte puntual de la lógica, normalmente un service, aislando dependencias externas.

****Prueba de integración:**** Verifica que varias partes funcionen juntas, especialmente el acceso real a la base de datos.

****Prueba de controller:**** Verifica que un endpoint responda correctamente en términos de URL, código HTTP y estructura de respuesta.

****Testcontainers:**** Herramienta que permite levantar contenedores temporales de Docker durante las pruebas, por ejemplo para usar una base de datos PostgreSQL real.

****Mock:**** Objeto simulado usado en pruebas unitarias para reemplazar dependencias reales, como repositorios o servicios externos.

****Sustentación:**** Espacio en el que el grupo debe explicar, defender y justificar el proyecto frente a los profesores.



1. Contexto del problema y propósito del proyecto

El proyecto nace de una necesidad concreta: organizar de forma correcta la atención médica dentro de un entorno universitario. No se trata simplemente de “hacer un backend por hacer uno”, sino de resolver un problema real de coordinación, disponibilidad, control y trazabilidad en la gestión de citas médicas.

En instituciones donde la atención se maneja de forma manual o dispersa, aparecen errores que no solo afectan la organización del sistema, sino también la experiencia del paciente, el uso de los consultorios y la capacidad de tomar decisiones con base en datos. Este backend busca precisamente dar una solución a ese problema, centralizando la gestión y aplicando reglas de negocio que evitan errores frecuentes.

1.1. Problema real que busca resolver la API

El problema principal es la mala gestión de las citas médicas cuando no existe un sistema centralizado y confiable que controle pacientes, doctores, consultorios, horarios y reservas en tiempo real.

Cuando las citas se manejan de forma manual, con hojas de cálculo, mensajes sueltos o herramientas dispersas, es muy fácil cometer errores. Un doctor puede terminar con dos pacientes asignados al mismo tiempo. Un consultorio puede quedar reservado para dos citas en el mismo bloque horario. Un paciente puede ser agendado en dos lugares a la vez. También pueden programarse citas por fuera del horario real del profesional, o incluso registrarse reservas en fechas pasadas sin sentido operativo.

Además, cuando no existe trazabilidad, la institución no sabe con claridad cuántas citas se cancelan, cuántas se completan, cuántos pacientes faltan, qué consultorios se usan más o qué tan productivos son los doctores. Eso limita mucho la capacidad de organizar mejor el servicio de salud universitario.

La API fue construida precisamente para eliminar ese tipo de problemas. No solo permite registrar información, sino que obliga a que cada operación pase por validaciones reales



antes de ser aceptada. Es decir, no se limita a “guardar datos”: controla que esos datos tengan sentido desde el punto de vista del negocio.

1.2. Por qué este problema importa en un contexto universitario

En un entorno universitario, la atención médica suele ser un servicio transversal que impacta a estudiantes, docentes y personal administrativo. Por eso, un problema de organización en las citas no es un detalle menor: puede afectar directamente la calidad del servicio, la disponibilidad de recursos y la confianza en la institución.

Si la gestión de citas falla, los pacientes pueden perder tiempo, los doctores pueden tener cargas mal distribuidas y los consultorios pueden usarse de forma ineficiente. También se vuelve más difícil priorizar la atención, entender qué especialidades tienen más demanda y detectar patrones como inasistencias frecuentes o uso excesivo de ciertos espacios físicos.

A nivel administrativo, esto también importa porque la universidad necesita información confiable para tomar decisiones. No basta con saber que “se están haciendo citas”; también es necesario medir ocupación, productividad, cancelaciones y no-shows. Sin un sistema bien diseñado, esas métricas quedan incompletas o dependen de trabajo manual, lo cual es poco confiable y difícil de sostener.

Por eso el problema no es solo técnico. Es un problema organizacional y operativo. El backend del proyecto aporta valor porque convierte un proceso potencialmente caótico en un flujo controlado, auditable y medible.

1.3. Qué evita el sistema (duplicidad, traslapes, mala trazabilidad, etc.)

Uno de los mayores aportes del sistema es que no permite que la lógica de las citas dependa únicamente de la memoria o del cuidado humano. En lugar de eso, traduce las reglas del negocio a validaciones concretas dentro del software.

Primero, evita la duplicidad y los traslapes. Un doctor no puede tener dos citas activas en el mismo horario. Un consultorio no puede estar ocupado por dos reservas que se crucen. Un paciente no puede quedar agendado simultáneamente en dos citas incompatibles. Estas



tres validaciones son fundamentales porque atacan directamente el problema más común de una agenda mal organizada.

Segundo, evita citas fuera del horario permitido. El sistema no acepta una cita simplemente porque la hora “parece libre”; también verifica que esa cita esté completamente dentro del horario laboral configurado para el doctor en el día correspondiente. Esto evita que se programen citas imposibles o incoherentes.

Tercero, evita inconsistencias de estado. No cualquier cita puede confirmarse, cancelarse o completarse en cualquier momento. El sistema controla la máquina de estados de la cita para que el flujo tenga sentido. Por ejemplo, una cita no puede completarse si todavía no ha llegado su hora de inicio, y una cita ya terminada no debería volver a estados anteriores.

Cuarto, evita datos inconsistentes en los módulos base. No deberían existir dos pacientes con el mismo correo o la misma matrícula universitaria, ni dos doctores con la misma licencia, ni dos especialidades o tipos de cita con el mismo nombre si deben ser únicos. El sistema también protege esas partes del dominio.

Quinto, evita la mala trazabilidad. Al registrar correctamente los estados de las citas y permitir consultas y reportes, el sistema deja evidencia de lo que ocurre: qué se agenda, qué se confirma, qué se cancela, qué se completa y quién no asiste. Eso convierte el proyecto en una herramienta útil no solo para operar, sino también para analizar.

1.4. Qué logra el sistema al final

Al final, el sistema logra convertir un proceso potencialmente desorganizado en una gestión de citas mucho más estructurada, confiable y medible.

Desde el punto de vista funcional, permite registrar pacientes, doctores, especialidades, consultorios, tipos de cita y horarios de atención. También permite consultar disponibilidad real, crear citas médicas, moverlas entre estados válidos y generar reportes de operación.

Desde el punto de vista técnico, logra separar bien las responsabilidades del proyecto: una parte recibe solicitudes, otra aplica reglas, otra consulta la base de datos y otra transforma la información para devolver respuestas claras. Esto mejora la organización interna del software y facilita su mantenimiento.



Desde el punto de vista del negocio, el sistema aporta orden. Impide errores frecuentes de agenda, mejora el aprovechamiento de consultorios, controla la trazabilidad del ciclo de vida de las citas y ofrece datos útiles para la toma de decisiones.

Y desde el punto de vista académico, el proyecto demuestra el uso de un stack moderno de backend con arquitectura por capas, persistencia relacional, validaciones reales, consultas avanzadas y pruebas automatizadas. Es decir, no solo cumple una función operativa, sino que también refleja una implementación seria y defendible frente a la rúbrica del taller.

2. Qué construyó exactamente el equipo

El equipo construyó un backend de tipo API REST para la gestión de citas médicas universitarias. El sistema no incluye interfaz gráfica propia, sino que está diseñado para recibir solicitudes HTTP y responder con información en formato JSON. Su función principal es centralizar y controlar el ciclo de vida completo de las citas médicas dentro de una clínica universitaria, evitando errores de agenda, validando reglas de negocio y generando información útil para la operación y la toma de decisiones.

La solución fue desarrollada como una aplicación monolítica organizada por capas. Esto significa que el proyecto separa claramente la entrada de peticiones, la lógica de negocio, el acceso a la base de datos, la transformación de datos y el manejo de errores. Esa organización no solo hace que el sistema funcione, sino que también permite entenderlo, mantenerlo y probarlo con mayor facilidad.

2.1. Tipo de sistema entregado

El sistema entregado es una API REST backend construida para administrar la reserva y gestión de consultorios médicos universitarios. Su propósito es permitir que diferentes operaciones relacionadas con la atención médica puedan ejecutarse de forma controlada desde el servidor, sin depender de procesos manuales ni de herramientas desconectadas entre sí.



Desde el punto de vista técnico, se trata de una aplicación que expone endpoints HTTP para crear, consultar y actualizar información. Los clientes externos, como Postman, archivos de prueba HTTP o un posible frontend futuro, interactúan con la API enviando solicitudes como GET, POST o PUT. La aplicación procesa esas solicitudes, valida reglas del negocio y responde con datos estructurados.

Desde el punto de vista funcional, no es un simple CRUD de tablas. Aunque sí permite registrar y consultar datos básicos, el verdadero valor del sistema está en su lógica: calcula automáticamente la hora de finalización de una cita, valida que no existan traslapes, verifica que los recursos estén activos, controla los cambios de estado de la cita y permite generar reportes operativos.

En resumen, el equipo entregó un sistema backend serio, orientado a resolver un problema institucional concreto, con estructura académicamente sólida y con suficiente profundidad técnica para ser sustentado.

2.2. Qué módulos funcionales incluye

El proyecto está organizado en varios módulos funcionales que representan las piezas principales del dominio de la clínica universitaria.

El primer grupo de módulos corresponde a la información base del sistema. Aquí están los pacientes, los doctores, las especialidades, los consultorios y los tipos de cita. Estos módulos permiten registrar y administrar los elementos fundamentales con los que luego se construyen las citas médicas.

El módulo de pacientes permite crear, consultar y actualizar la información de las personas que reciben atención. El módulo de doctores permite registrar profesionales de la salud, asociarlos a una especialidad y mantener su información básica. El módulo de especialidades funciona como catálogo de áreas médicas disponibles. El módulo de consultorios administra los espacios físicos donde se prestan los servicios. El módulo de tipos de cita define categorías de atención y su duración estándar en minutos.

El segundo grupo corresponde a la gestión de horarios. Aquí aparece el módulo de horarios de doctores, que permite registrar las franjas horarias de atención de cada profesional según el día de la semana. Este módulo es esencial, porque el sistema usa esos horarios como base para validar si una cita cabe realmente dentro de la agenda del médico.



El tercer grupo es el núcleo del proyecto: las citas médicas. Este módulo permite crear citas, consultarlas, filtrarlas por distintos criterios y moverlas dentro de una máquina de estados controlada. Una cita puede ser agendada, confirmada, cancelada, completada o marcada como no asistida. Cada transición está regulada por reglas específicas del negocio.

El cuarto grupo es el de disponibilidad. Este módulo no solo muestra horarios registrados, sino que calcula los huecos reales disponibles de un doctor para una fecha determinada, teniendo en cuenta tanto sus franjas de trabajo como las citas activas ya existentes.

El quinto grupo es el de reportes. El proyecto incluye reportes de ocupación de consultorios, productividad de doctores y pacientes con mayor cantidad de no show. Estos reportes permiten observar el comportamiento operativo del sistema y respaldan decisiones administrativas.

Visto en conjunto, el sistema cubre la parte operativa principal de una agenda médica universitaria: catálogos base, horarios, reservas, disponibilidad y análisis.

2.3. Qué usuarios o roles implícitos interactúan con el sistema

Aunque el proyecto no implementa autenticación ni manejo formal de roles, sí está construido alrededor de varios roles implícitos del negocio. Estos roles no existen como usuarios técnicos dentro del código, pero sí están presentes en la lógica del sistema y en la forma en que se entiende su uso.

El primer rol implícito es el administrador o personal de gestión. Este sería quien registra pacientes, doctores, especialidades, consultorios, tipos de cita y horarios. También puede consultar la información base del sistema y mantenerla actualizada.

El segundo rol implícito es el usuario operativo de agenda o recepción. Este rol es el que crea citas, las confirma, las cancela o consulta disponibilidad para encontrar un espacio válido dentro de la agenda del médico.

El tercer rol implícito es el profesional de salud. Aunque el sistema no le asigna un inicio de sesión ni una interfaz propia, su presencia es clara en la lógica de completar citas y en la



productividad reportada por el sistema. En términos del negocio, es quien termina asociado a la atención y al resultado de la cita.

El cuarto rol implícito es el coordinador o responsable administrativo. Este rol se relaciona especialmente con la consulta de reportes, porque necesita medir ocupación de consultorios, productividad médica y comportamiento de inasistencias.

El quinto rol implícito es el paciente, no como operador directo del backend, sino como entidad central de la atención. El sistema guarda su información, protege sus conflictos de agenda y registra si asistió o no a la cita.

En otras palabras, aunque el proyecto no modela usuarios autenticados, sí refleja con claridad los actores reales del proceso y sus necesidades dentro del dominio.

2.4. Qué cosas sí cubre el proyecto y qué cosas no cubre

El proyecto sí cubre de manera clara y defendible la gestión principal de una clínica universitaria en lo relacionado con agenda médica. Permite registrar pacientes, doctores, especialidades, consultorios, tipos de cita y horarios. Permite consultar disponibilidad real de doctores en una fecha. Permite crear citas con validaciones fuertes, confirmar citas, cancelarlas con motivo, completarlas con observaciones opcionales y marcarlas como no asistidas. También permite consultar reportes operativos relevantes para la administración.

Además, el sistema sí cubre varias reglas importantes que lo elevan por encima de un CRUD simple. Valida que los actores existan y estén activos, impide citas en el pasado, verifica que una cita esté dentro del horario del doctor, calcula automáticamente la hora de finalización según la duración del tipo de cita y evita traslapes de doctor, consultorio y paciente. También controla correctamente la máquina de estados de las citas.

Desde el punto de vista técnico, sí cubre arquitectura por capas, persistencia relacional, consultas agregadas, manejo de errores, DTOs separados de entidades, mappers y pruebas automatizadas de varios niveles.

Lo que no cubre el proyecto también debe decirse con claridad. No implementa autenticación ni autorización con usuarios reales o roles de seguridad. No tiene frontend propio. No incluye paginación avanzada en listados. No maneja eliminación física de



registros como operación principal del negocio. Tampoco incorpora notificaciones, agenda externa, integración con correo o sistemas clínicos externos.

Esto no significa que el proyecto esté incompleto para el taller. Significa que su enfoque está en resolver bien el backend funcional exigido por la rúbrica, no en cubrir todo el ecosistema de una plataforma hospitalaria completa.

3. Tecnologías usadas y qué aporta cada una

El proyecto fue construido con un conjunto de tecnologías que responden directamente al stack exigido por el taller y a las necesidades reales del sistema. Cada una cumple una función específica dentro de la solución, y entender su papel ayuda mucho en la sustentación.

La idea no es que un compañero memorice detalles avanzados de cada herramienta, sino que sepa por qué fue usada y qué aporta dentro del proyecto. En una sustentación, esa explicación vale mucho más que repetir nombres sueltos de librerías.

3.1. Java 21

Java 21 es el lenguaje base sobre el que se construyó todo el proyecto. Es la tecnología central del backend y el punto de partida de la solución. Todas las clases principales del sistema, como entidades, controllers, services, repositories, DTOs, mappers y pruebas, fueron escritas en Java.

Su aporte principal al proyecto es servir como lenguaje robusto, tipado y ampliamente usado en backend empresarial. Gracias a Java, el proyecto puede modelar entidades complejas, separar capas, aplicar validaciones y estructurar la lógica de negocio de manera ordenada.

Para el equipo, Java 21 también representa el marco académico exigido por el taller. Es decir, no fue una elección casual, sino parte del stack obligatorio. Además, al trabajar con Spring Boot y JPA, Java permite integrar de forma natural la lógica del negocio con la persistencia y las pruebas.



En palabras simples, Java es el idioma en el que está escrito todo el sistema.

3.2. Spring Boot 4

Spring Boot 4 es el framework principal del proyecto. Su función es simplificar la construcción del backend y proporcionar la infraestructura necesaria para que la API funcione como aplicación web.

Gracias a Spring Boot, el proyecto puede levantar un servidor embebido, registrar rutas HTTP, inyectar dependencias entre clases, manejar configuraciones y organizar la aplicación en componentes bien definidos. Es el framework que hace posible que existan controllers, services, repositories y manejo global de errores de manera integrada.

Una de sus grandes ventajas es que reduce mucha configuración manual. En lugar de construir una aplicación web desde cero, Spring Boot ofrece una base lista para desarrollar la lógica del sistema y exponerla como API REST.

Dentro del proyecto, Spring Boot es lo que permite que una ruta como `/api/appointments` exista, reciba datos, los procese y responda correctamente.

3.3. PostgreSQL

PostgreSQL es el motor de base de datos relacional usado por el sistema. Allí se persiste toda la información del proyecto: pacientes, doctores, especialidades, consultorios, tipos de cita, horarios y citas médicas.

Su aporte principal es almacenar de forma estructurada y confiable los datos del sistema, respetando relaciones, llaves foráneas y restricciones de integridad. Esto es especialmente importante en un proyecto como este, donde una cita conecta varias entidades y no puede quedar inconsistente.

PostgreSQL también es importante porque el proyecto depende de consultas reales para detectar traslapes, calcular reportes y mantener la integridad de la agenda. No se trata solo de guardar registros, sino de consultarlos con precisión y de forma agregada.



En resumen, PostgreSQL es la base relacional que sostiene el modelo de datos del sistema.

3.4. Spring Data JPA

Spring Data JPA es la tecnología que conecta el código Java con la base de datos relacional. Su función es permitir que las entidades del proyecto se comporten como representación de tablas y que los repositories puedan consultar y guardar información sin escribir SQL manual para todo.

Con JPA, las clases `Patient`, `Doctor`, `Office`, `Appointment` y demás se mapean a tablas de la base de datos. A la vez, los repositories pueden heredar operaciones estándar como buscar por id, listar todo o guardar registros, y además definir consultas más avanzadas cuando se necesita lógica específica.

En este proyecto, Spring Data JPA aporta muchísimo porque facilita tanto operaciones CRUD normales como consultas personalizadas para overlaps, disponibilidad y reportes. Esto permite una capa repository potente y ordenada, muy alineada con lo que pide la rúbrica.

Dicho de forma simple, Spring Data JPA es el puente entre el mundo del código Java y el mundo de la base de datos.

3.5. Testcontainers

Testcontainers es una herramienta usada para levantar contenedores temporales durante las pruebas. En este proyecto su papel es muy importante porque permite ejecutar pruebas de integración contra una base de datos PostgreSQL real dentro de Docker.

Eso significa que el proyecto no se limita a probar la lógica con simulaciones. También valida que las consultas, relaciones y comportamientos reales funcionen correctamente en una base de datos auténtica. Esto da mucha más confianza en el sistema y fortalece la capa de testing.



Para la sustentación, esta tecnología es valiosa porque demuestra que el proyecto no solo fue probado con mocks, sino también con una infraestructura real y aislada para pruebas.

En términos sencillos, Testcontainers sirve para crear una base de datos temporal y limpia cada vez que se ejecutan ciertos tests.

3.6. JUnit 5

JUnit 5 es el framework principal de pruebas del proyecto. Se usa para escribir y ejecutar tests automáticos, organizarlos y verificar que el sistema se comporte como se espera.

Su aporte dentro del proyecto es permitir expresar de forma clara escenarios de prueba como crear una cita válida, rechazar una cita en el pasado, validar traslapes o comprobar transiciones de estado. Es la base sobre la que se construyen tanto las pruebas unitarias como parte de las de integración.

Para el equipo, JUnit 5 representa la herramienta principal para demostrar que la lógica implementada no solo existe, sino que fue validada.

En resumen, JUnit 5 es el motor de las pruebas automatizadas del proyecto.

3.7. Mockito

Mockito es una herramienta complementaria a JUnit usada principalmente en pruebas unitarias. Su función es crear objetos simulados, también llamados mocks, que reemplazan dependencias reales como repositorios o servicios externos.

Esto permite probar la lógica de una clase sin necesitar una base de datos real en cada caso. Por ejemplo, se puede simular que un repositorio devuelve un paciente activo o que reporta un conflicto de horario, y así verificar cómo responde el service frente a esas condiciones.



Su aporte al proyecto es aislar la lógica de negocio para probarla más rápido y de forma controlada. Es una pieza clave para los tests unitarios de services como ``AppointmentServiceImpl``, ``PatientServiceImpl`` o ``DoctorServiceImpl``.

En pocas palabras, Mockito ayuda a probar comportamiento sin depender de todo el sistema real alrededor.

3.8. Lombok

Lombok es una librería usada para reducir código repetitivo. Su función principal es generar automáticamente partes comunes de las clases, como getters, setters, constructores, builders y otros elementos de apoyo.

En un proyecto como este, con muchas entidades y DTOs, escribir a mano todos esos métodos haría el código mucho más largo y repetitivo. Lombok permite que las clases sean más limpias y que el equipo se concentre en la lógica importante en lugar de en el boilerplate.

Su aporte no está en cambiar el funcionamiento del sistema, sino en hacer el código más compacto y más fácil de mantener.

Dicho de manera simple, Lombok sirve para quitar trabajo repetitivo y dejar el código más ordenado.

3.9. Mermaid documentación y requests.http

Además de las tecnologías principales del backend, el proyecto también usa herramientas de apoyo documental y operativo.

Mermaid se usa para representar visualmente el modelo entidad relación en el documento MER. Su aporte es facilitar la comprensión de cómo se conectan las entidades del sistema, lo cual es muy útil tanto para documentación como para sustentación.



El archivo ``README.md`` aporta una visión general del proyecto, su stack, arquitectura, reglas y forma de ejecución. Es un documento de entrada para entender la solución.

El archivo ``MER.md`` organiza el modelo de datos y ayuda a explicar las entidades, atributos y relaciones. Es especialmente útil cuando se quiere defender el diseño relacional.

El archivo ``requests.http`` funciona como colección de ejemplos de peticiones para probar la API. Su aporte es práctico porque permite ver rápidamente cómo se llaman los endpoints, qué datos reciben y qué partes del sistema están expuestas por HTTP.

En conjunto, estos recursos complementan el código y facilitan tanto el estudio como la demostración del proyecto.

4. Arquitectura general del proyecto

La arquitectura general del proyecto es uno de los aspectos más importantes para entender cómo está construido el sistema. No basta con saber que la API funciona. En una sustentación también es clave explicar por qué el código está organizado de cierta manera y qué ventaja tiene esa organización.

Este proyecto sigue una arquitectura por capas. Eso quiere decir que no mezcla todas las responsabilidades en un solo lugar, sino que divide el trabajo en partes claras. Una capa recibe la petición, otra aplica la lógica del negocio, otra consulta la base de datos, otra transforma la información y otra se encarga de manejar errores.

Esta separación es muy útil porque hace que el sistema sea más legible, más mantenible y más fácil de probar. Además, encaja muy bien con lo que normalmente se espera en un backend construido con Spring Boot.

4.1. Qué significa arquitectura por capas

Arquitectura por capas significa que el sistema está dividido en niveles o bloques de responsabilidad, y que cada uno cumple una función específica. En lugar de tener una sola



clase que haga todo, el proyecto distribuye el trabajo para que cada capa se encargue de un tipo de tarea.

En este sistema, la capa de controller recibe las solicitudes HTTP. La capa de service contiene la lógica del negocio. La capa de repository se comunica con la base de datos. Las entidades representan el modelo de datos. Los DTOs sirven para transportar información entre el sistema y el cliente. Los mappers convierten entidades en respuestas. Y la capa de excepciones controla cómo responder cuando algo falla.

La idea es que cada capa tenga una responsabilidad clara y no invada el trabajo de las demás. Así el proyecto se vuelve más comprensible y cada archivo tiene un propósito definido.

4.2. Flujo general de una petición dentro del proyecto

Cuando un cliente hace una solicitud a la API, la petición entra primero por un controller. Ese controller interpreta la ruta y el método HTTP, recibe los datos del body o de los parámetros y delega el trabajo al service correspondiente.

El service toma esa petición y aplica la lógica del negocio. Aquí es donde ocurren las validaciones realmente importantes. Por ejemplo, si se intenta crear una cita, el service verifica que existan las entidades necesarias, que estén activas, que la cita no esté en el pasado, que caiga dentro del horario del doctor y que no tenga conflictos de agenda.

Si el service necesita consultar o guardar información, entonces llama al repository. El repository se encarga de hablar con la base de datos. Puede usar operaciones estándar o consultas personalizadas.

Una vez que el repository devuelve la información o guarda los cambios, el service toma el resultado y, si es necesario, lo transforma en un formato más adecuado para responder. Allí entran los mappers y los DTOs de respuesta.

Finalmente, el controller devuelve la respuesta HTTP al cliente. Si en algún punto ocurrió un error, como un recurso no encontrado o una violación de una regla de negocio, entra en acción el manejador global de excepciones para devolver un mensaje claro con el código HTTP adecuado.



En resumen, el flujo general es este: el controller recibe, el service decide, el repository consulta o guarda, el mapper transforma y el exception handler responde correctamente si algo falla.

4.3. Cómo se relacionan controller service repository y base de datos

La relación entre estas piezas puede entenderse como una cadena ordenada de responsabilidades.

El controller es la puerta de entrada de la API. Es el punto donde llegan las solicitudes externas. Su trabajo es recibir la petición, validar la forma básica de los datos y delegar la operación. El controller no debería tomar decisiones de negocio complejas.

El service es el cerebro operativo del sistema. Allí viven las reglas del negocio. Es la capa que entiende qué significa realmente crear una cita, cancelarla o calcular disponibilidad. El service decide qué validaciones ejecutar, en qué orden y qué hacer si algo no cumple las reglas.

El repository es la capa que se comunica directamente con la base de datos. No decide si una operación tiene sentido de negocio, sino que ejecuta consultas y operaciones de persistencia. Su fortaleza está en buscar, filtrar, guardar y agrupar datos.

La base de datos es el lugar donde se almacena la información del proyecto. Allí están representadas las entidades principales del dominio y sus relaciones.

La relación entre estas partes no es horizontal ni caótica. Es una relación ordenada. El controller depende del service. El service depende del repository. El repository depende de la base de datos. Esa estructura hace que el sistema sea más claro, porque evita que una capa haga el trabajo de otra.

4.4. Por qué esta arquitectura ayuda a organizar el proyecto



Esta arquitectura ayuda porque separa los problemas del sistema en piezas manejables. Si un compañero quiere entender las reglas del negocio, sabe que debe mirar la capa service. Si quiere ver qué rutas expone la API, puede ir a los controllers. Si quiere ver cómo se consultan los datos, puede revisar los repositories. Si quiere entender la estructura de la información, puede estudiar las entidades.

También mejora el mantenimiento. Si cambia una regla de negocio, normalmente basta con modificar un service. Si cambia una respuesta de la API, probablemente habrá que tocar un DTO o un mapper. Si aparece una nueva consulta, se puede agregar en el repository sin mezclar todo en un solo archivo gigante.

Además, esta forma de organizar el proyecto facilita mucho las pruebas. Como la lógica está concentrada en los services, es posible probarla de manera unitaria usando mocks. Como las consultas están en los repositories, también pueden validarse con pruebas de integración sobre una base de datos real. Y como los controllers están separados, se pueden probar a nivel HTTP sin tocar toda la lógica interna.

Finalmente, esta arquitectura es defendible académicamente porque refleja buenas prácticas de diseño backend. No solo hace que el sistema funcione, sino que demuestra orden, claridad y separación de responsabilidades, que son justamente cosas que se valoran en un proyecto como este.

5. Estructura de carpetas del proyecto

La estructura de carpetas del proyecto es una de las claves para entender cómo está organizado el sistema. En una sustentación no basta con decir que el proyecto funciona. También es importante demostrar que el código está bien distribuido, que cada carpeta tiene un propósito claro y que la solución no fue construida de forma desordenada.

Este proyecto sigue una organización muy típica de aplicaciones backend en Spring Boot, basada en capas y responsabilidades. Eso significa que las carpetas no están agrupadas al azar, sino según la función que cumplen dentro del sistema. Hay carpetas para el modelo de datos, para la lógica de negocio, para el acceso a la base de datos, para los endpoints, para los objetos de entrada y salida, para los convertidores y para el manejo de errores. Además, existe una estructura paralela para las pruebas.

Comprender esta estructura ayuda mucho a estudiar el sistema porque permite responder preguntas como estas:



Dónde se definen las entidades del negocio
Dónde están los endpoints de la API
Dónde viven las reglas de negocio
Dónde se hacen las consultas complejas
Dónde están las pruebas más importantes

5.1. Vista general del árbol principal

En la raíz del proyecto están los archivos principales de configuración, documentación y apoyo. Allí se encuentran el archivo pom.xml, que define dependencias y proceso de construcción, y los archivos documentales como README.md, MER.md y requests.http. A partir de esa raíz aparece la carpeta src, que contiene el código fuente principal y el código de pruebas.

Dentro de src/main/java/com/university/clinic/ se concentra la aplicación real. Allí están las carpetas por capas como entity, repository, service, service/impl, controller, dto, mapper y exception. Cada una agrupa archivos que cumplen una función específica dentro del backend.

Dentro de src/main/resources/ está la configuración de la aplicación, en especial el archivo application.yml, que define parámetros como puerto, conexión a base de datos y comportamiento de JPA.

Dentro de src/test/java/com/university/clinic/ está toda la parte de pruebas. Esa sección se divide principalmente en pruebas de controller, de service e integración. También existe un test de contexto general que valida que la aplicación arranque correctamente.

La idea general del árbol del proyecto puede entenderse así:

Raíz del proyecto
Documentación y configuración general
src/main/java
Código fuente principal organizado por capas
src/main/resources
Configuración de ejecución
src/test/java
Pruebas automatizadas organizadas por tipo

Esta vista global es importante porque permite ubicar rápidamente cualquier parte del sistema.

5.2. entity/

La carpeta entity/ contiene las clases que representan el modelo principal del negocio. En términos simples, aquí están definidas las entidades que luego se convierten en tablas dentro de la base de datos.



Cada archivo de esta carpeta representa una parte importante del dominio. Por ejemplo, aquí viven los pacientes, los doctores, las especialidades, los consultorios, los tipos de cita, los horarios del doctor y las citas médicas. También están los enums que controlan ciertos estados del sistema, como el estado de una cita, el estado de un paciente o el estado de un consultorio.

Esta carpeta es importante porque aquí se ve la estructura central del sistema. Si alguien quiere entender qué información guarda el proyecto y cómo se relaciona, debe empezar por esta carpeta.

En esta sección también aparece `BaseEntity`, que sirve como clase base común para varias entidades y aporta campos compartidos como identificador, fecha de creación y fecha de actualización.

En resumen, `entity/` responde a esta pregunta:
Qué cosas existen dentro del sistema y cómo están modeladas

5.3. repository/

La carpeta `repository/` contiene las interfaces que se encargan de comunicarse con la base de datos. Su trabajo principal es consultar, guardar y filtrar información.

Aquí es donde se concentran los accesos a datos. Algunos repositories son relativamente simples y aprovechan métodos estándar heredados de JPA, como buscar por id, listar todos o guardar un registro. Sin embargo, otros, especialmente el de citas, tienen consultas más avanzadas para cubrir reglas importantes del sistema.

Esta carpeta es muy importante porque es el puente entre el modelo del negocio y la persistencia real en PostgreSQL. Además, aquí viven varias consultas clave del proyecto, como la validación de traslapes, la búsqueda de horarios por día, la consulta de citas por rango de fechas y los reportes agregados.

En resumen, `repository/` responde a esta pregunta:
Cómo accede el sistema a la información almacenada en la base de datos

5.4. dto/

La carpeta `dto/` contiene los objetos de transferencia de datos. Son clases diseñadas para recibir o devolver información a través de la API sin exponer directamente las entidades internas del sistema.

Aquí aparecen los objetos que representan lo que el cliente envía cuando quiere crear o actualizar algo, y también los objetos que el sistema devuelve como respuesta. Esto ayuda a proteger el modelo interno y a controlar mejor la forma de los datos.

Por ejemplo, cuando alguien crea una cita no envía una entidad completa de Java. Envía un DTO de entrada con campos específicos. Del mismo modo, cuando el sistema responde, construye un DTO de salida más limpio y controlado.



Esta carpeta es importante porque separa la estructura interna de la base de datos de la forma externa en que la API se comunica con sus clientes.

En resumen, dto/ responde a esta pregunta:
Qué datos entran y salen del sistema a través de la API

5.5. service/

La carpeta service/ contiene las interfaces de servicio del proyecto. Estas interfaces definen qué operaciones puede realizar cada módulo del sistema, pero no explican todavía cómo se implementan.

Su función principal es declarar el contrato de cada servicio. Por ejemplo, allí se define que el servicio de citas puede crear, confirmar, cancelar, completar, marcar no show o consultar por distintos filtros.

Esta carpeta es importante porque separa la definición de las operaciones de su implementación concreta. Eso ayuda a mantener una arquitectura más limpia y facilita el testing y el desacoplamiento.

En resumen, service/ responde a esta pregunta:
Qué puede hacer cada servicio del sistema en términos generales

5.6. service/impl/

La carpeta service/impl/ contiene las implementaciones reales de los servicios. Esta es una de las carpetas más importantes del proyecto porque aquí vive la lógica de negocio.

En esta sección se toman las decisiones importantes del sistema. Es aquí donde se valida si una cita puede crearse, si un doctor está activo, si el consultorio está disponible, si una transición de estado es válida o si un paciente tiene un conflicto de agenda.

Mientras la carpeta service/ define el contrato, la carpeta service/impl/ resuelve el comportamiento real. Por eso, cuando alguien quiere entender cómo funciona el sistema de verdad, esta carpeta es una parada obligatoria.

Dentro de esta carpeta destacan especialmente las implementaciones relacionadas con citas, disponibilidad, horarios de doctores y reportes, porque concentran la mayor complejidad del proyecto.

En resumen, service/impl/ responde a esta pregunta:
Dónde están las reglas de negocio reales del sistema

5.7. controller/

La carpeta controller/ contiene los controladores de la API. Estos archivos son el punto de entrada del sistema desde el exterior. Allí se definen las rutas HTTP que los clientes pueden consumir y la forma en que esas rutas delegan el trabajo al servicio correspondiente.



Cada controller agrupa endpoints de un módulo funcional. Por ejemplo, hay controllers para pacientes, doctores, especialidades, consultorios, tipos de cita, horarios, citas, disponibilidad y reportes.

Su función no es contener lógica de negocio compleja, sino recibir la solicitud, interpretar parámetros, validar la estructura básica del request y llamar al service correspondiente.

Esta carpeta es clave para la sustentación porque es la que permite mostrar cómo se expone funcionalmente el sistema al exterior.

En resumen, controller/ responde a esta pregunta:
Qué endpoints tiene la API y a qué módulo pertenece cada uno

5.8. mapper/

La carpeta mapper/ contiene las clases encargadas de transformar entidades del sistema en DTOs de respuesta. Su función principal es adaptar los datos internos para que la API devuelva información clara, controlada y útil.

Esto es importante porque una entidad de la base de datos no siempre debe enviarse directamente al cliente. A veces hace falta convertir relaciones, concatenar nombres, ocultar detalles internos o estructurar mejor la respuesta.

Los mappers ayudan a mantener la separación entre el modelo interno y la representación externa de la API. Además, mejoran la claridad del código porque centralizan la conversión de datos en clases específicas.

En resumen, mapper/ responde a esta pregunta:
Cómo se transforman los datos internos del sistema en respuestas listas para la API

5.9. exception/

La carpeta exception/ contiene las clases relacionadas con el manejo de errores. Aquí viven tanto las excepciones personalizadas como el manejador global que traduce los errores a respuestas HTTP legibles.

Esta sección es importante porque permite que el sistema no responda con errores crudos o confusos. En su lugar, organiza diferentes tipos de fallos según la naturaleza del problema. Por ejemplo, una entidad inexistente no se trata igual que un conflicto de horario o que una violación de una regla de negocio.

Gracias a esta carpeta, el proyecto puede responder con códigos HTTP adecuados y mensajes claros cuando algo sale mal.

En resumen, exception/ responde a esta pregunta:
Cómo controla el sistema los errores y cómo los comunica al cliente



5.10. src/test/java/

La carpeta de pruebas contiene todos los tests automatizados del proyecto. Está organizada para validar diferentes capas y comportamientos del sistema.

Dentro de esta sección existen pruebas unitarias para services, pruebas de integración para repositories y pruebas de controller para endpoints HTTP. También hay una prueba de contexto general para verificar que la aplicación arranque sin problemas.

Esta carpeta es especialmente importante para la sustentación porque demuestra que el proyecto no solo fue construido, sino también probado. Además, permite explicar cómo se validó la lógica del negocio, cómo se comprobó el acceso real a la base de datos y cómo se verificó el comportamiento de la API.

En resumen, src/test/java/ responde a esta pregunta: Cómo se comprobó que el sistema funciona correctamente

5.11. Archivos raíz importantes: README.md, MER.md, requests.http, pom.xml

Además de las carpetas principales, en la raíz del proyecto existen varios archivos muy importantes que ayudan a entender, ejecutar, probar y documentar la solución.

pom.xml es el archivo de configuración de Maven. Allí se definen las dependencias del proyecto, el stack técnico, el plugin de Spring Boot y la versión de Java. Es la referencia principal para saber con qué tecnologías fue construido el backend.

README.md es el documento general del proyecto. Su objetivo es explicar de forma resumida qué hace el sistema, qué tecnologías usa, cómo está organizado y cómo puede ejecutarse.

MER.md documenta el modelo entidad relación del sistema. Es especialmente útil para entender cómo se conectan las entidades y cuál es la estructura relacional del proyecto.

requests.http funciona como colección de peticiones para probar la API. Contiene ejemplos de endpoints, parámetros y bodies, lo que facilita tanto la demostración práctica como la comprensión funcional del sistema.

Estos archivos son importantes porque complementan el código y permiten entrar al proyecto desde distintos ángulos: técnico, documental y funcional.

6. Archivos más importantes por carpeta y qué hace cada uno

Una vez entendida la estructura general de carpetas, el siguiente paso lógico es identificar cuáles son los archivos más importantes dentro de cada una y qué papel cumplen. Esto es



muy útil para estudiar, porque no todos los archivos tienen el mismo peso dentro del proyecto.

Aunque el sistema tiene varias clases y cada una cumple su función, hay ciertos archivos que concentran la mayor parte de la lógica, del valor funcional o de la comprensión del proyecto. Son esos archivos los que un compañero debería priorizar si tiene poco tiempo para prepararse.

En esta sección no se busca listar absolutamente todos los archivos del proyecto con el mismo nivel de detalle, sino resaltar los más importantes por categoría y explicar por qué valen la pena.

6.1. Archivos clave de entity/

Dentro de entity/, el archivo más importante es Appointment.java. Esta clase representa la cita médica y es la entidad central del sistema. Su importancia radica en que conecta al paciente, al doctor, al consultorio y al tipo de cita. Además, es la entidad sobre la que recaen las validaciones más fuertes, la lógica del ciclo de vida y buena parte de los reportes.

Después de Appointment.java, otro archivo clave es DoctorSchedule.java. Este archivo define los horarios de atención de los doctores y es fundamental para dos cosas: validar si una cita cae dentro del horario permitido y calcular la disponibilidad real de un médico en una fecha determinada.

También es muy importante Doctor.java, porque representa a uno de los actores principales del sistema y se relaciona tanto con la especialidad como con las citas y los horarios. De forma similar, Patient.java es importante porque representa al usuario que recibe atención y porque sus conflictos de agenda también deben controlarse.

Office.java es importante porque modela el recurso físico del consultorio y permite controlar si un espacio está disponible o en mantenimiento. AppointmentType.java también es clave porque define la duración estándar de cada tipo de cita y permite calcular automáticamente el campo endAt.

Specialty.java tiene un papel más de catálogo, pero sigue siendo importante para entender cómo se clasifica a los doctores dentro del dominio.

Finalmente, BaseEntity.java es un archivo muy importante desde el punto de vista técnico porque centraliza campos comunes como el identificador y las marcas de tiempo. Ayuda a entender por qué varias entidades comparten una estructura base.

En esta carpeta, también merecen atención los enums AppointmentStatus, PatientStatus y OfficeStatus, porque explican los estados controlados del sistema y son fundamentales para la lógica de negocio.

6.2. Archivos clave de repository/

En la carpeta repository/, el archivo más importante es AppointmentRepository.java. Este es, con diferencia, el repositorio más complejo del sistema. Su importancia radica en que



no se limita a operaciones simples de guardar y consultar, sino que contiene varias consultas personalizadas necesarias para que la lógica del proyecto funcione.

Aquí se encuentran las consultas que verifican si existe traslape de doctor, de consultorio o de paciente. También están las consultas que permiten buscar citas activas de un doctor para calcular disponibilidad y las consultas agregadas que alimentan los reportes de ocupación, productividad e inasistencias.

Por eso, AppointmentRepository.java es clave para entender cómo el sistema traslada reglas complejas del negocio a consultas reales sobre la base de datos.

Otro archivo importante es DoctorScheduleRepository.java, porque permite consultar los horarios de un doctor por día de la semana o listarlos en general. Sin este repositorio, no podría validarse correctamente si una cita encaja en el horario del médico ni podría calcularse disponibilidad.

También son importantes PatientRepository.java y DoctorRepository.java porque incorporan validaciones preventivas de unicidad, como correo, matrícula o licencia profesional. Estas validaciones son menos complejas que las del módulo de citas, pero siguen siendo muy relevantes para la calidad del sistema.

SpecialtyRepository.java y AppointmentTypeRepository.java cumplen un papel similar en relación con la unicidad de nombres dentro de catálogos.

En general, esta carpeta es clave para entender cómo el sistema habla con la base de datos y cómo ciertas decisiones del negocio dependen de consultas concretas.

6.3. Archivos clave de service/impl/

Dentro de service/impl/, el archivo más importante es AppointmentServiceImpl.java. Este archivo puede considerarse el corazón del proyecto. Allí se concentra la lógica principal relacionada con las citas médicas, que es el eje más fuerte de toda la solución.

En esta clase se valida la existencia y el estado de las entidades relacionadas con la cita, se controla que la fecha no esté en el pasado, se calcula automáticamente la hora de finalización, se valida el horario del doctor, se detectan traslapes y se controlan las transiciones entre estados como confirmar, cancelar, completar o marcar no show.

Si un compañero entiende bien AppointmentServiceImpl.java, ya tiene dominada una parte enorme de la sustentación.

El segundo archivo más importante es AvailabilityServiceImpl.java. Este archivo es clave porque contiene la lógica que calcula los slots libres de un doctor en una fecha. No se limita a leer horarios guardados, sino que cruza esos horarios con las citas existentes y determina qué espacios realmente están disponibles.

El tercer archivo más importante es ReportServiceImpl.java. Su papel es tomar los resultados crudos de las consultas agregadas del repository y convertirlos en reportes útiles para la API. Es importante porque muestra una parte más analítica del sistema.



Después de esos tres, también son relevantes `DoctorScheduleServiceImpl.java`, `PatientServiceImpl.java`, `DoctorServiceImpl.java`, `SpecialtyServiceImpl.java` y `AppointmentTypeServiceImpl.java`, porque controlan validaciones de sus respectivos módulos, especialmente en temas de solapamiento de horarios o prevención de duplicados.

6.4. Archivos clave de controller/

En la carpeta `controller/`, el archivo más importante es `AppointmentController.java`. Esto se debe a que es el controller más completo y el que expone el flujo principal del sistema. Allí están los endpoints para crear citas, consultarlas, filtrarlas por distintos criterios y moverlas entre estados.

Este controller es clave porque sirve como demostración práctica de la funcionalidad central del backend. Además, muestra varios patrones importantes, como el uso de filtros por parámetros, rutas específicas para transiciones de estado y soporte para operaciones complejas sobre una entidad principal.

Otro controller muy importante es `AvailabilityController.java`, porque expone la funcionalidad de cálculo de disponibilidad, que es una de las más interesantes del proyecto desde el punto de vista algorítmico.

También destaca `ReportController.java`, ya que concentra los endpoints de análisis y métricas del sistema. Es importante para explicar cómo el proyecto no solo administra datos, sino que también produce información útil para la gestión.

Los demás controllers como `PatientController`, `DoctorController`, `SpecialtyController`, `OfficeController`, `AppointmentTypeController` y `DoctorScheduleController` son igualmente importantes para comprender el alcance completo del sistema, aunque su lógica suele ser más directa y menos compleja que la de citas, disponibilidad o reportes.

6.5. Archivos clave de exception/

Dentro de `exception/`, el archivo más importante es `GlobalExceptionHandler.java`. Este archivo tiene un papel fundamental porque organiza la forma en que el sistema responde cuando ocurre un error.

Su importancia está en que evita respuestas caóticas o poco entendibles. En lugar de devolver fallos crudos, centraliza el tratamiento de errores y los traduce a respuestas HTTP con códigos y mensajes coherentes.

También son importantes `ResourceNotFoundException.java`, `BusinessException.java`, `ConflictException.java` y `ValidationException.java`. Aunque son archivos pequeños, representan categorías clave de errores del sistema.

`ResourceNotFoundException` explica el caso en que no se encuentra un recurso solicitado. `BusinessException` representa violaciones de reglas del negocio. `ConflictException` se usa para choques o conflictos de datos, especialmente en agenda. `ValidationException` sirve para validaciones personalizadas.



Estos archivos son pequeños pero muy valiosos para entender que el proyecto no solo ejecuta operaciones, sino que también sabe responder correctamente cuando algo no puede hacerse.

6.6. Archivos clave de src/test/

Dentro de las pruebas, el archivo más importante es `AppointmentServiceImplTest.java`. Esto se debe a que cubre la lógica principal del proyecto y valida varios escenarios relacionados con la creación de citas y la máquina de estados. Es probablemente el mejor archivo para estudiar si se quiere entender tanto la lógica como la forma en que fue probada.

Otro archivo muy importante es `AppointmentRepositoryIntegrationTest.java`, porque demuestra que el sistema no solo fue probado con simulaciones, sino también con una base de datos real en pruebas de integración. Esto respalda con fuerza la capa repository y las consultas complejas.

`AvailabilityServiceImplTest.java` también es clave porque valida el algoritmo de cálculo de disponibilidad, una de las partes más interesantes del sistema. `DoctorScheduleServiceImplTest.java` es importante porque protege la lógica de horarios, especialmente los solapamientos entre franjas de un mismo doctor.

`ReportServiceImplTest.java` tiene valor porque comprueba que los reportes se construyen correctamente a partir de resultados agregados. A nivel HTTP, `AppointmentControllerTest.java` es muy importante porque demuestra que los endpoints principales de citas responden correctamente en diferentes escenarios. `ReportControllerTest.java` también ayuda a respaldar la parte analítica.

Por su parte, `PatientServiceImplTest.java`, `DoctorServiceImplTest.java`, `SpecialtyServiceImplTest.java`, `AppointmentTypeServiceImplTest.java` y `OfficeServiceImplTest.java` son importantes para mostrar que incluso los módulos más sencillos fueron probados, especialmente en lo relacionado con duplicados y actualizaciones parciales.

Finalmente, `ClinicReservationsApplicationTest.java` es valioso porque demuestra que el contexto general de la aplicación puede arrancar correctamente.

En conjunto, esta carpeta de pruebas es una de las mayores fortalezas del proyecto y un excelente respaldo para la sustentación.

7. Modelo de datos explicado en lenguaje sencillo

El modelo de datos del proyecto representa las piezas principales que necesita una clínica universitaria para gestionar citas médicas de manera organizada. No se trata simplemente de guardar registros aislados, sino de construir una red de información en la que varias entidades se relacionan entre sí para que una cita tenga sentido completo.



La mejor forma de entender este modelo es pensar que el sistema necesita responder preguntas muy concretas. Quién es el paciente. Qué doctor lo atiende. En qué consultorio será atendido. Qué tipo de cita se realizará. Cuánto dura. En qué horario trabaja el doctor. Y en qué estado se encuentra la cita.

Por eso el proyecto no se apoya en una sola tabla grande ni en datos sueltos. Está compuesto por siete entidades principales que, al relacionarse, permiten construir la lógica real del sistema. Entre todas ellas, la entidad más importante es la cita médica, porque es el punto donde convergen los demás elementos del dominio.

7.1. Las 7 entidades obligatorias

El sistema está construido alrededor de siete entidades obligatorias que representan la base del dominio.

La primera entidad es Patient. Esta representa al paciente, es decir, a la persona que solicita o recibe la atención médica.

La segunda entidad es Doctor. Esta representa al profesional de la salud que atiende la cita.

La tercera entidad es Specialty. Esta representa la especialidad médica a la que pertenece cada doctor, por ejemplo medicina general, cardiología u odontología.

La cuarta entidad es Office. Esta representa el consultorio físico donde se realiza la atención.

La quinta entidad es AppointmentType. Esta representa el tipo de cita y define la duración estándar de la atención.

La sexta entidad es DoctorSchedule. Esta representa los horarios de trabajo de un doctor según el día de la semana.



La séptima entidad es Appointment. Esta representa la cita médica en sí misma y es la entidad central del sistema.

Estas siete entidades no existen de forma aislada. Juntas construyen la estructura lógica que permite registrar disponibilidad, crear citas, controlar traslapes y generar reportes.

7.2. Qué representa cada entidad

Patient representa al paciente que será atendido. En el contexto universitario, puede tratarse de un estudiante, un docente o un miembro del personal. El sistema necesita guardar su identidad básica, sus medios de contacto, su matrícula universitaria y su estado operativo.

Doctor representa al médico o profesional responsable de la atención. Además de sus datos básicos, el sistema necesita saber a qué especialidad pertenece y si se encuentra activo para recibir citas.

Specialty representa el catálogo de especialidades médicas disponibles dentro de la clínica. Su función principal es clasificar a los doctores y organizar la oferta de atención.

Office representa el consultorio físico. No es un dato decorativo. Es un recurso real del negocio y por eso el sistema debe controlar si está disponible, inactivo o en mantenimiento.

AppointmentType representa la categoría de la cita. Su papel más importante es definir cuántos minutos dura una atención de ese tipo. Gracias a eso el sistema puede calcular automáticamente la hora de finalización de una cita.

DoctorSchedule representa los horarios de trabajo de un doctor. Indica en qué días atiende y entre qué horas. Esta entidad es fundamental para validar si una cita cabe dentro del marco laboral del médico y para calcular disponibilidad.

Appointment representa la cita médica concreta. Aquí se conectan el paciente, el doctor, el consultorio y el tipo de cita dentro de un intervalo de tiempo específico. También se registra su estado y sus observaciones. Esta es la entidad más importante del sistema porque concentra la lógica principal del proyecto.



7.3. Relaciones entre entidades

La relación más sencilla de entender es la que existe entre Specialty y Doctor. Una especialidad puede estar asociada a muchos doctores, pero cada doctor pertenece a una sola especialidad.

La relación entre Doctor y DoctorSchedule también es clara. Un doctor puede tener varios horarios configurados, porque puede atender en distintos días y franjas horarias. Sin embargo, cada horario pertenece a un solo doctor.

La relación entre Patient y Appointment indica que un paciente puede tener muchas citas a lo largo del tiempo, pero cada cita pertenece a un solo paciente.

La relación entre Doctor y Appointment indica que un doctor puede atender muchas citas, pero cada cita queda asignada a un solo doctor.

La relación entre Office y Appointment muestra que un consultorio puede usarse en muchas citas distintas en diferentes momentos, pero cada cita ocurre en un solo consultorio.

La relación entre AppointmentType y Appointment significa que un tipo de cita puede aplicarse a muchas citas reales, pero cada cita tiene un solo tipo asociado.

La entidad Appointment es la que une todas estas relaciones. Por eso se considera el centro del modelo. Sin ella, las demás entidades serían solo catálogos o configuraciones estáticas. Con ella, el sistema se vuelve operativo.

En términos simples, las relaciones se pueden resumir así

Una especialidad agrupa a muchos doctores

Un doctor tiene muchos horarios



Un paciente puede tener muchas citas

Un doctor puede atender muchas citas

Un consultorio puede albergar muchas citas

Un tipo de cita puede usarse en muchas citas

Cada cita pertenece a un solo paciente un solo doctor un solo consultorio y un solo tipo de cita

7.4. Campos más importantes de cada entidad

En Patient los campos más importantes son el nombre, el apellido, el correo, el teléfono, la matrícula universitaria y el estado. El correo y la matrícula son especialmente importantes porque deben ser únicos y sirven para evitar duplicados.

En Doctor los campos más importantes son el nombre, el apellido, el número de licencia, la especialidad y el indicador de si está activo. La licencia es clave porque no debe repetirse y funciona como identificador profesional.

En Specialty el campo más importante es el nombre. Aunque es una entidad sencilla, cumple una función estructural importante dentro del modelo.

En Office los campos más importantes son el nombre, el piso y el estado. El estado es clave porque determina si el consultorio puede recibir citas o si está fuera de servicio.

En AppointmentType los campos más importantes son el nombre y la duración en minutos. La duración es uno de los campos más relevantes del proyecto porque de ella depende el cálculo automático de la hora final de la cita.



En DoctorSchedule los campos más importantes son el doctor asociado, el día de la semana, la hora de inicio, la hora de fin y el estado activo. Estos campos permiten saber exactamente cuándo trabaja un médico.

En Appointment los campos más importantes son las referencias a paciente, doctor, consultorio y tipo de cita, además de la hora de inicio, la hora de finalización, el estado y las notas. `startAt` y `endAt` son esenciales para detectar traslapes y construir reportes. El estado permite seguir el ciclo de vida de la cita.

Además de estos campos particulares, varias entidades comparten identificador, fecha de creación y fecha de actualización gracias a la clase base común del proyecto.

7.5. Estados del sistema `AppointmentStatus`, `PatientStatus`, `OfficeStatus`

El proyecto usa enums para controlar estados del sistema de manera cerrada y segura. Eso significa que ciertas entidades no pueden tener cualquier valor arbitrario, sino solo los estados definidos formalmente.

AppointmentStatus controla el ciclo de vida de una cita. Sus valores son SCHEDULED, CONFIRMED, CANCELLED, COMPLETED y NO_SHOW.

SCHEDULED significa que la cita fue creada y quedó agendada.

CONFIRMED significa que la cita ya fue confirmada y está lista para ocurrir.

CANCELLED significa que la cita fue cancelada y ya no se realizará.

COMPLETED significa que la cita sí se llevó a cabo y fue atendida.

NO_SHOW significa que el paciente no asistió a la cita.



PatientStatus controla si un paciente está habilitado o no para participar en el sistema. Sus valores son ACTIVE e INACTIVE. Un paciente inactivo no debería recibir nuevas citas.

OfficeStatus controla la condición operativa de un consultorio. Sus valores son ACTIVE, INACTIVE y UNDER_MAINTENANCE. Esto permite distinguir entre un espacio disponible, uno fuera de servicio y uno temporalmente en mantenimiento.

Estos estados son importantes porque convierten reglas del negocio en decisiones formales del sistema. Gracias a ellos, el backend no solo guarda información, sino que también entiende cuándo una entidad puede o no participar en una operación.

8. Módulos funcionales del sistema

Los módulos funcionales del sistema representan las áreas principales de trabajo del backend. Esta forma de explicarlo es muy útil para la sustentación porque permite responder una pregunta esencial

Qué puede hacer el sistema

En lugar de mirar el proyecto por carpetas técnicas, aquí se mira por funcionalidad. Esto ayuda especialmente a compañeros que no dominan Java, porque primero entienden el comportamiento del sistema y después pueden conectar esa funcionalidad con el código.

Cada módulo responde a una necesidad del negocio y se relaciona con uno o varios controllers, services y repositories. Algunos módulos son más simples, como los catálogos base, y otros son más complejos, como citas, disponibilidad y reportes.

8.1. Pacientes



El módulo de pacientes permite registrar, consultar y actualizar la información de las personas que reciben atención médica. Su objetivo es mantener una base ordenada de usuarios del servicio de salud.

Dentro de este módulo se gestionan datos básicos como nombre, apellido, correo, teléfono, matrícula universitaria y estado del paciente. No se trata solo de guardar información, sino también de protegerla frente a duplicados. Por ejemplo, el sistema evita que dos pacientes se registren con el mismo correo o con la misma matrícula.

Este módulo es importante porque el paciente es uno de los actores centrales del sistema. Sin una entidad paciente bien controlada, no tendría sentido el resto de la lógica de citas.

8.2. Doctores

El módulo de doctores permite registrar, consultar y actualizar a los profesionales de la salud que atienden dentro de la clínica universitaria.

Aquí se guarda información como nombre, apellido, número de licencia, estado activo y especialidad asociada. Una de las reglas importantes del módulo es que la licencia profesional no puede repetirse.

Este módulo también conecta directamente con la lógica de citas, disponibilidad y reportes. Un doctor no es solo un dato de catálogo, sino un actor central cuya actividad y horarios afectan el funcionamiento completo del sistema.

8.3. Especialidades

El módulo de especialidades funciona como catálogo de áreas médicas. Permite crear, consultar y actualizar las distintas ramas de atención disponibles en la clínica.

Su función es organizar a los doctores dentro de una clasificación médica coherente. Aunque es un módulo sencillo, es importante porque da estructura al dominio y evita que los doctores existan sin contexto profesional.



También protege la unicidad del nombre de la especialidad, lo cual ayuda a mantener consistencia dentro del sistema.

8.4. Consultorios

El módulo de consultorios administra los espacios físicos donde se realizan las atenciones médicas. Permite crear, consultar y actualizar consultorios.

Cada consultorio tiene nombre, piso y estado. El estado es especialmente importante porque un consultorio puede estar activo, inactivo o en mantenimiento. Esto impacta directamente la posibilidad de asignarle citas.

Este módulo convierte al consultorio en un recurso controlado por el sistema y no en un simple dato decorativo. Eso es clave para la lógica de agenda.

8.5. Tipos de cita

El módulo de tipos de cita administra las categorías de atención médica y la duración estándar asociada a cada una. Permite crear, consultar y actualizar tipos de cita.

Su función principal es parametrizar el tiempo de atención. Esto significa que una cita no necesita recibir manualmente su hora de finalización, porque el sistema la calcula a partir de la duración definida en este módulo.

Es un módulo pequeño pero muy importante para una de las reglas principales del proyecto.

8.6. Horarios de doctores

El módulo de horarios de doctores permite registrar y consultar las franjas horarias de atención de cada médico según el día de la semana.



Este módulo es fundamental porque el sistema no permite crear citas simplemente porque haya espacio teórico en la agenda. También exige que la cita esté completamente dentro de un horario real configurado para el doctor.

Además, este módulo protege contra el solapamiento de horarios de un mismo doctor en el mismo día. Por eso no es solo un catálogo de horarios, sino una parte crítica de la coherencia del sistema.

8.7. Citas médicas

El módulo de citas médicas es el núcleo del proyecto. Permite crear, consultar, filtrar y cambiar el estado de las citas.

Aquí es donde convergen el paciente, el doctor, el consultorio y el tipo de cita. También es donde se ejecutan las validaciones más importantes del sistema. Antes de crear una cita, el backend verifica que las entidades existan, que estén activas, que la fecha no esté en el pasado, que la cita caiga dentro del horario del doctor y que no existan traslapes.

Además, este módulo controla la máquina de estados de la cita. Una cita puede ser confirmada, cancelada, completada o marcada como no asistida, pero solo siguiendo reglas válidas.

Este es el módulo que mejor representa la profundidad real del proyecto.

8.8. Disponibilidad

El módulo de disponibilidad permite calcular los espacios libres de un doctor en una fecha determinada.

No se limita a leer los horarios registrados. Lo que hace es cruzar los horarios del doctor con las citas activas ya existentes y devolver únicamente los slots realmente disponibles.

Este módulo es muy valioso porque muestra una lógica más algorítmica dentro del sistema y permite resolver una necesidad real de agenda.



En términos funcionales, es el módulo que responde a la pregunta

Qué horas libres tiene este doctor en esta fecha para una cita de cierta duración

8.9. Reportes

El módulo de reportes permite analizar el comportamiento operativo del sistema a través de métricas y consultas agregadas.

Incluye tres reportes principales

Reporte de ocupación de consultorios

Reporte de productividad de doctores

Reporte de pacientes con mayor número de no show

Este módulo es importante porque eleva el proyecto más allá de la gestión operativa y lo convierte también en una herramienta de análisis. No solo organiza citas, sino que permite observar cómo se está usando el sistema.

9. Endpoints principales y qué hace cada uno

Los endpoints son las rutas mediante las cuales el sistema expone su funcionalidad al exterior. Son una parte muy importante de la sustentación porque permiten responder de forma directa qué puede hacer la API y cómo se usa.



En este proyecto, los endpoints están distribuidos por módulos funcionales y siguen una lógica REST bastante clara. Algunos se usan para crear o consultar información base, otros para operar la agenda y otros para obtener reportes.

No es necesario memorizar todas las rutas del sistema, pero sí conviene dominar las más importantes y entender qué hace cada una.

9.1. Tabla resumen de endpoints por módulo

Pacientes

GET `/api/patients`

Lista todos los pacientes

GET `/api/patients/{id}`

Busca un paciente por id

POST `/api/patients`

Crea un paciente nuevo

PUT `/api/patients/{id}`

Actualiza un paciente existente

Doctores

GET `/api/doctors`

Lista todos los doctores

GET `/api/doctors/{id}`

Busca un doctor por id



Informe técnico — Proyecto Web

Versión: 1

POST `/api/doctors`

Crea un doctor nuevo

PUT `/api/doctors/{id}`

Actualiza un doctor existente

Especialidades

GET `/api/specialties`

Lista todas las especialidades

GET `/api/specialties/{id}`

Busca una especialidad por id

POST `/api/specialties`

Crea una especialidad nueva

PUT `/api/specialties/{id}`

Actualiza una especialidad existente

Consultorios

GET `/api/offices`

Lista todos los consultorios

GET `/api/offices/{id}`

Busca un consultorio por id

POST `/api/offices`

Crea un consultorio nuevo



PUT `/api/offices/{id}`

Actualiza un consultorio existente

Tipos de cita

GET `/api/appointment-types`

Lista los tipos de cita

GET `/api/appointment-types/{id}`

Busca un tipo de cita por id

POST `/api/appointment-types`

Crea un tipo de cita nuevo

PUT `/api/appointment-types/{id}`

Actualiza un tipo de cita existente

Horarios de doctores

GET `/api/doctors/{doctorId}/schedules`

Lista los horarios del doctor

POST `/api/doctors/{doctorId}/schedules`

Crea una franja horaria para el doctor

Citas médicas

GET `/api/appointments`

Lista todas las citas

GET `/api/appointments/{id}`



Busca una cita por id

GET `/api/appointments?doctorId=...`

Filtra citas por doctor

GET `/api/appointments?patientId=...`

Filtra citas por paciente

GET `/api/appointments?status=...`

Filtra citas por estado

GET `/api/appointments?from=...&to=...`

Filtra citas por rango de fechas

POST `/api/appointments`

Crea una cita nueva

PUT `/api/appointments/{id}/confirm`

Confirma una cita

PUT `/api/appointments/{id}/cancel`

Cancela una cita

PUT `/api/appointments/{id}/complete`

Completa una cita

PUT `/api/appointments/{id}/no-show`

Marca una cita como no asistida

Disponibilidad



GET `/api/availability/doctors/{doctorId}?date=...&durationMinutes=...`

Calcula los slots libres del doctor en una fecha determinada

Reportes

GET `/api/reports/office-occupancy?from=...&to=...`

Devuelve métricas de ocupación por consultorio

GET `/api/reports/doctor-productivity?from=...&to=...`

Devuelve métricas de productividad por doctor

GET `/api/reports/no-show-patients?from=...&to=...`

Devuelve pacientes con mayor número de inasistencias

9.2. Endpoints más importantes que sí o sí deben saberse

Aunque el proyecto tiene varios endpoints, hay algunos que son especialmente importantes para la sustentación porque concentran la complejidad real del sistema.

El primero es `POST /api/appointments`. Este endpoint es probablemente el más importante de todo el proyecto porque demuestra la creación de una cita con todas sus validaciones de negocio.

El segundo es `PUT /api/appointments/{id}/confirm`, porque muestra que la cita tiene un ciclo de vida y que el sistema controla la transición desde agendada hasta confirmada.

El tercero es `PUT /api/appointments/{id}/complete`, porque permite explicar que completar una cita no es solo cambiar un estado, sino validar que ya haya llegado el momento correcto y permitir observaciones opcionales.

El cuarto es `PUT /api/appointments/{id}/cancel`, porque demuestra cancelación controlada con motivo.



El quinto es `PUT /api/appointments/{id}/no-show`, porque permite mostrar el registro formal de inasistencia.

El sexto es `GET /api/availability/doctors/{doctorId}?date=...&durationMinutes=...`, porque muestra la lógica de disponibilidad real.

El séptimo conjunto importante son los tres endpoints de reportes, porque permiten defender la parte analítica del proyecto.

Si un compañero domina esos endpoints, ya tiene una base muy fuerte para explicar el valor real del sistema.

9.3. Ejemplos sencillos de uso

Ejemplo de creación de especialidad

POST `/api/specialties`

Body

```
```json
{
 "name": "Cardiología"
}
```
```

Ejemplo de creación de doctor

POST `/api/doctors`

Body



```
``json
{
  "firstName": "Carlos",
  "lastName": "Rodríguez",
  "licenseNumber": "X998877",
  "specialtyId": 1
}
...

```

Ejemplo de creación de paciente

POST `/api/patients`

Body

```
``json
{
  "firstName": "Ana",
  "lastName": "Gómez",
  "universityId": "11223344",
  "email": "ana.gomez@mail.com",
  "phone": "5558765"
}
...

```

Ejemplo de creación de tipo de cita

POST `/api/appointment-types`

Body

```
``json
{

```



```
"name": "Chequeo General",  
"durationMinutes": 30  
}  
...
```

Ejemplo de creación de horario de doctor

POST `/api/doctors/1/schedules`

Body

```
``json  
{  
  "dayOfWeek": "MONDAY",  
  "startTime": "08:00:00",  
  "endTime": "14:00:00"  
}  
...
```

Ejemplo de consulta de disponibilidad

GET `/api/availability/doctors/1?date=2026-04-20&durationMinutes=30`

Ejemplo de creación de cita

POST `/api/appointments`

Body

```
``json  
{  
  "patientId": 1,  
  "doctorId": 1,
```



Informe técnico — Proyecto Web

Versión: 1

```
"officeId": 1,  
"appointmentTypeId": 1,  
"startAt": "2026-04-20T09:00:00",  
"notes": "Dolor de cabeza leve"  
}  
...
```

Ejemplo de confirmar cita

PUT `/api/appointments/1/confirm`

Ejemplo de completar cita con observaciones

PUT `/api/appointments/1/complete`

Body

```
``json  
{  
  "notes": "El paciente se retiró con fórmula médica"  
}  
...
```

Ejemplo de cancelar cita

PUT `/api/appointments/1/cancel`

Body

```
``json  
{  
  "reason": "El doctor tuvo una urgencia familiar"  
}
```



...

Ejemplo de reporte de ocupación

GET ``/api/reports/office-occupancy?from=2026-04-01T00:00:00&to=2026-04-30T23:59:59``

Ejemplo de reporte de productividad

GET ``/api/reports/doctor-productivity?from=2026-04-01T00:00:00&to=2026-04-30T23:59:59``

Ejemplo de reporte de no show

GET ``/api/reports/no-show-patients?from=2026-04-01T00:00:00&to=2026-04-30T23:59:59``

9.4. Códigos HTTP más comunes del proyecto

El sistema usa varios códigos HTTP que es importante entender en la sustentación.

200 OK se usa cuando una operación se ejecuta correctamente y la respuesta devuelve información o confirma un cambio válido.

201 Created se usa cuando se crea un recurso nuevo, por ejemplo un paciente, un doctor o una cita.

400 Bad Request se usa cuando la petición tiene errores de validación en su estructura o en campos obligatorios.

404 Not Found se usa cuando se busca un recurso que no existe.



409 Conflict se usa cuando la operación entra en conflicto con reglas del sistema, por ejemplo en casos de duplicados o traslapes.

422 Unprocessable Entity se usa cuando la petición está bien formada, pero viola una regla de negocio, como intentar crear una cita en el pasado o completar una cita antes de tiempo.

500 Internal Server Error representa errores inesperados del sistema, aunque idealmente no debería ser la respuesta habitual en un flujo normal.

Entender estos códigos es importante porque ayudan a demostrar que la API no solo hace operaciones, sino que también comunica correctamente el resultado de cada situación al cliente.

10. Reglas de negocio más importantes del proyecto

Las reglas de negocio son el conjunto de condiciones que el sistema debe respetar para que una operación tenga sentido dentro del contexto de una clínica universitaria. En este proyecto, la mayoría de las reglas más importantes están concentradas en la lógica del módulo de citas médicas, porque es allí donde convergen paciente, doctor, consultorio, tipo de cita, horario y estado de la atención.

Estas reglas son importantes porque impiden que el sistema se convierta en un simple registro de datos sin control. Gracias a ellas, la API no solo guarda información, sino que decide si una acción está permitida o no según las necesidades reales del negocio.

10.1. Reglas al crear una cita

La creación de una cita es la operación más sensible del sistema, porque implica validar varias condiciones antes de aceptar la reserva.

La primera regla es que el paciente debe existir. No puede crearse una cita para una persona que no esté registrada en el sistema.

La segunda regla es que el doctor debe existir. Tampoco se puede asignar una cita a un profesional inexistente.

La tercera regla es que el consultorio debe existir. El sistema no permite reservar un espacio que no esté registrado.



La cuarta regla es que el tipo de cita debe existir. Esto es necesario porque de allí sale la duración estándar de la atención.

La quinta regla es que el paciente debe estar activo. Si un paciente se encuentra inactivo, no debería participar en nuevas reservas.

La sexta regla es que el doctor debe estar activo. Un médico deshabilitado o fuera de servicio no puede recibir citas.

La séptima regla es que el consultorio debe estar disponible en estado operativo. Si el consultorio está inactivo o en mantenimiento, el sistema no debe permitir asignarle una cita.

La octava regla es que la cita no puede estar en el pasado. No tiene sentido operativo crear una atención en una fecha y hora que ya pasaron.

La novena regla es que la hora final de la cita no la define el cliente, sino el servidor. El sistema calcula endAt automáticamente sumando la duración del tipo de cita a la hora inicial.

La décima regla es que la cita debe caer completamente dentro del horario de trabajo del doctor. No basta con que la hora “parezca libre”. Debe existir una franja horaria válida configurada para ese día.

La undécima regla es que el doctor no puede tener otra cita activa que se cruce con la nueva. Esta es una de las validaciones más importantes del proyecto.

La duodécima regla es que el consultorio no puede estar ocupado por otra cita activa en el mismo rango.

La decimotercera regla es que el paciente tampoco puede tener otra cita activa que choque con la nueva.

En conjunto, estas reglas garantizan que una cita solo se cree si todos los recursos y condiciones del negocio están alineados correctamente.

10.2. Reglas para confirmar una cita

Confirmar una cita no significa simplemente cambiar un estado. También está controlado por reglas del sistema.

La regla principal es que solo una cita en estado agendado puede pasar a estado confirmado. Esto evita que una cita ya cancelada, ya completada o ya marcada como no asistida vuelva a entrar al flujo como si estuviera pendiente.

La confirmación sirve para indicar que la cita ya fue aceptada o ratificada y que sigue activa dentro del ciclo normal del servicio.

En términos del negocio, esta regla protege la coherencia de la máquina de estados y evita transiciones ilógicas.



10.3. Reglas para cancelar una cita

La cancelación también está controlada por reglas claras.

La primera regla es que solo pueden cancelarse citas que estén en estado agendado o confirmado. Si una cita ya fue completada o ya quedó como no asistida, no debería poder cancelarse después.

La segunda regla es que la cancelación debe registrar un motivo. Esto es importante porque deja trazabilidad y ayuda a entender por qué una cita no se realizó.

La tercera regla es que el motivo no se pierde. El sistema lo agrega a las notas de la cita, de modo que quede registro de la razón de cancelación.

Desde el punto de vista del negocio, cancelar una cita también libera la agenda porque esa cita deja de contar como activa para futuras validaciones de conflicto.

10.4. Reglas para completar una cita

Completar una cita es una transición más estricta que confirmar o cancelar.

La primera regla es que solo una cita confirmada puede pasar a completada. Si una cita nunca fue confirmada, no debería marcarse como atendida.

La segunda regla es que la cita no puede completarse antes de su hora de inicio. Esta validación es importante porque evita que el sistema registre atenciones que todavía no ocurren.

La tercera regla es que, si se envían observaciones finales, estas pueden agregarse al registro de la cita. Esto permite guardar notas administrativas o de cierre sin alterar la lógica principal.

Estas reglas garantizan que el estado completado signifique realmente que la atención ya sucedió.

10.5. Reglas para marcar no-show

El no-show representa una inasistencia del paciente y por eso también debe seguir reglas estrictas.

La primera regla es que solo una cita confirmada puede marcarse como no-show. No tendría sentido marcar como ausente una cita que ni siquiera había sido ratificada.

La segunda regla es que la hora de inicio ya debe haber pasado. No se puede declarar ausente a un paciente antes de que llegue el momento real de la cita.

Esta lógica es importante porque protege la seriedad del registro de inasistencias y evita falsos positivos dentro del sistema.



10.6. Reglas de disponibilidad

La disponibilidad no se calcula tomando únicamente el horario del doctor. El sistema aplica varias reglas para que los slots libres sean reales.

La primera regla es que el doctor debe existir. Si no existe, no puede hablarse de disponibilidad.

La segunda regla es que la disponibilidad se calcula sobre los horarios activos del doctor para un día específico de la semana. Eso significa que el sistema no inventa espacios, sino que parte de las franjas que ya fueron configuradas.

La tercera regla es que solo las citas activas bloquean espacio. En la práctica, eso significa que las citas agendadas o confirmadas ocupan tiempo, mientras que una cita cancelada deja de bloquear ese intervalo.

La cuarta regla es que la duración solicitada importa. No es lo mismo buscar huecos de treinta minutos que de una hora. El sistema genera slots según la duración solicitada y luego revisa cuáles caben realmente sin cruzarse con citas ya registradas.

La quinta regla es que un slot solo se considera disponible si no se solapa con ninguna cita activa del doctor en ese rango de tiempo.

En conjunto, estas reglas hacen que la disponibilidad sea una proyección realista y no una simple lista de horas teóricas.

10.7. Reglas de reportes

Los reportes del proyecto también siguen reglas específicas, aunque de un tipo distinto. No se trata de validar actores o horarios, sino de calcular métricas correctas a partir de la información existente.

La primera regla es que los reportes trabajan por rango de fechas. Esto permite acotar el análisis a un periodo útil para la gestión.

La segunda regla es que los datos no se devuelven crudos, sino agrupados y transformados. Por ejemplo, el sistema no responde con cada cita individual cuando se pide productividad, sino con indicadores agregados por doctor.

La tercera regla es que los porcentajes se calculan con base en relaciones entre conteos reales. Por ejemplo, la ocupación de consultorios se apoya en la proporción entre citas completadas y total de citas del rango.

La cuarta regla es que los reportes deben reflejar estados reales del sistema, como completadas, canceladas o no show. Sin esa clasificación, las métricas perderían valor.

La quinta regla es que la información agregada debe devolverse en un formato comprensible para la API, no como una estructura interna cruda de base de datos.

Estas reglas hacen que el módulo de reportes no sea solo una consulta de datos, sino una capa de análisis útil para la operación.



11. El corazón del proyecto: flujo completo de una cita

El flujo de una cita médica es el núcleo del proyecto porque resume casi toda la lógica importante del sistema. Si una persona entiende bien cómo nace una cita, cómo se valida, cómo cambia de estado y cómo termina, entonces ya comprende la parte más fuerte del backend.

El valor de esta sección está en traducir el funcionamiento técnico a una historia clara. La idea es que cualquier integrante del grupo pueda explicar una cita desde su creación hasta su cierre, sin necesidad de hablar en lenguaje demasiado técnico.

11.1. Qué pasa cuando se crea una cita

Cuando se crea una cita, el cliente envía una solicitud al endpoint correspondiente con los datos básicos necesarios. Entre esos datos están el paciente, el doctor, el consultorio, el tipo de cita, la fecha y hora de inicio y, de forma opcional, una nota inicial.

Lo primero que ocurre es que el sistema recibe esa solicitud y valida que la estructura del request esté bien formada. Si faltan campos obligatorios, la petición se rechaza antes de llegar a la lógica profunda.

Luego, el sistema empieza a revisar si las entidades relacionadas realmente existen. Busca al paciente, al doctor, al consultorio y al tipo de cita. Si alguna de estas piezas no está registrada, la operación se detiene.

Después de eso, valida que los actores estén en condiciones de participar. El paciente debe estar activo, el doctor debe estar activo y el consultorio debe estar en un estado que permita recibir atención.

Una vez superadas esas validaciones, el sistema verifica que la cita no esté en el pasado. Si la fecha ya pasó, la operación no tiene sentido y se rechaza.

Luego calcula automáticamente la hora final usando la duración del tipo de cita. Ese cálculo permite construir el intervalo completo de tiempo que ocupará la reserva.

Con ese intervalo, el sistema revisa si la cita cae dentro del horario configurado para el doctor en ese día y luego verifica que no existan choques con otras citas del doctor, del consultorio o del paciente.

Solo si todas esas condiciones se cumplen, la cita se guarda y nace en estado agendado.

11.2. Cómo se valida si hay conflicto

La validación de conflicto es uno de los puntos más importantes del proyecto. No se trata de una sola comprobación, sino de tres conflictos distintos.



El primero es el conflicto del doctor. El sistema pregunta si el doctor ya tiene otra cita activa que se cruce con el nuevo intervalo. Si la respuesta es sí, la cita debe rechazarse.

El segundo es el conflicto del consultorio. Aquí la pregunta es si el espacio físico ya está ocupado por otra cita en ese mismo rango de tiempo. Si lo está, tampoco se puede crear la nueva reserva.

El tercer conflicto es el del paciente. El sistema revisa si esa persona ya tiene otra cita activa en el mismo horario. Si existe, la nueva cita también se rechaza.

Estas tres validaciones convierten el backend en una agenda coherente. Sin ellas, el proyecto podría guardar citas, pero no garantizaría una operación realista.

11.3. Cómo se calcula endAt

El cálculo de endAt es una decisión importante del diseño del sistema. El cliente no envía directamente la hora de finalización de la cita. En lugar de eso, el cliente envía la hora de inicio y el tipo de cita.

El sistema toma la duración estándar asociada al tipo de cita y la suma a la hora de inicio. Así obtiene automáticamente la hora final.

Esta regla tiene varias ventajas. Evita errores manuales, mantiene consistencia entre citas del mismo tipo y facilita las validaciones de traslape y los cálculos de reportes.

En términos del negocio, endAt no es un dato libre. Es una consecuencia lógica de la combinación entre inicio y duración.

11.4. Cómo cambia el estado de una cita

Una cita no permanece siempre igual. El sistema le asigna un ciclo de vida controlado mediante estados.

Cuando una cita nace, su estado inicial es agendado. Eso significa que la reserva ya fue creada, pero todavía no se ha confirmado.

Desde allí, puede pasar a confirmada si la operación de confirmación es válida. Una vez confirmada, la cita puede seguir tres caminos posibles: completarse, cancelarse o marcarse como no asistida.

No cualquier cambio de estado está permitido. El sistema impide transiciones ilógicas, como confirmar dos veces, completar una cita que nunca fue confirmada o cancelar una cita ya terminada.

Esta lógica convierte el estado de la cita en una pieza de negocio real y no en una etiqueta decorativa.



11.5. Qué pasa si se cancela

Si se cancela una cita, el sistema primero verifica que la cancelación esté permitida para el estado actual. Si la cita está en un estado válido para cancelar, cambia el estado a cancelada y registra el motivo.

Ese motivo queda almacenado en las notas de la cita. Esto es importante porque aporta trazabilidad y permite saber por qué no se realizó la atención.

Además, una vez cancelada, la cita deja de comportarse como una reserva activa. En términos prácticos, eso significa que libera el espacio temporal para futuras disponibilidades y deja de bloquear agenda.

La cancelación, por tanto, no es solo un cambio de palabra. Tiene impacto real sobre la operación del sistema.

11.6. Qué pasa si se completa

Cuando una cita se completa, el sistema verifica primero que la transición sea válida y que ya haya llegado el momento correcto para registrar la atención como realizada.

Si la cita cumple las condiciones, el estado cambia a completada. Si además se reciben observaciones finales, estas se adjuntan al contenido de notas.

Este paso es importante porque formaliza que la atención sí ocurrió y permite conservar un pequeño rastro administrativo de cierre.

Una cita completada se convierte en una evidencia de atención exitosa dentro del sistema y también en una pieza importante para los reportes posteriores.

11.7. Qué pasa si se marca como no asistida

Si el paciente no se presenta, el sistema puede marcar la cita como no asistida. Pero, al igual que en otras transiciones, esto no se hace libremente.

Primero debe tratarse de una cita confirmada y además la hora de inicio ya debe haber pasado. Si estas condiciones no se cumplen, la operación no debería aceptarse.

Cuando sí se cumplen, el estado cambia a no show y la cita queda registrada como una inasistencia formal. Esto tiene valor tanto operativo como analítico, porque luego puede utilizarse en reportes de comportamiento del paciente.

En otras palabras, el no show no es solo un estado más. Es una forma de registrar incumplimientos de atención y darles trazabilidad.



12. Cómo se calcula la disponibilidad

La disponibilidad es una de las partes más interesantes del proyecto porque demuestra que el sistema no solo guarda horarios, sino que también puede analizarlos y transformarlos en información útil.

Cuando alguien pregunta por la disponibilidad de un doctor en una fecha, el sistema no responde leyendo una tabla simple de horas. Lo que hace realmente es calcular cuáles espacios están libres teniendo en cuenta tanto el horario del médico como las citas que ya fueron reservadas.

Por eso este módulo tiene un valor especial dentro de la sustentación. Permite mostrar una lógica más dinámica y menos trivial.

12.1. Qué información se usa

Para calcular disponibilidad, el sistema necesita tres grupos principales de información.

El primero es la identidad del doctor. Es indispensable saber para qué profesional se quiere consultar la agenda.

El segundo es la fecha. A partir de ella, el sistema determina el día de la semana correspondiente y busca las franjas de trabajo del médico para ese día.

El tercero es la duración del slot que se quiere consultar. Esto permite ajustar el tamaño de los espacios que se van a generar y evaluar.

Además de eso, el sistema necesita consultar las citas activas que ese doctor ya tiene agendadas o confirmadas en esa fecha. Sin esa información no podría saber qué partes del horario ya están ocupadas.

En resumen, la disponibilidad se calcula cruzando doctor, fecha, duración solicitada, horarios del doctor y citas activas existentes.

12.2. Cómo se generan los slots

Una vez identificado el horario del doctor para el día consultado, el sistema toma cada franja y la divide en bloques de la duración solicitada.

Por ejemplo, si el doctor trabaja de ocho de la mañana a doce del mediodía y la duración pedida es de treinta minutos, entonces el sistema genera intervalos consecutivos de media hora dentro de esa franja.

Esos intervalos son los slots candidatos. Todavía no significa que estén libres, sino que son espacios posibles dentro del horario legal del doctor.

Después de generarlos, el sistema revisa uno por uno para decidir si realmente están disponibles o si deben descartarse porque se cruzan con citas existentes.



12.3. Qué hace que un slot esté libre u ocupado

Un slot está libre si cumple dos condiciones al mismo tiempo.

La primera es que esté dentro de una franja activa del horario del doctor en ese día. Si el espacio está fuera de las horas de trabajo configuradas, no puede considerarse disponible.

La segunda es que no se solape con ninguna cita activa del doctor en ese intervalo. Si existe una cita agendada o confirmada que se cruce con el slot, entonces ese espacio debe marcarse como ocupado.

Por el contrario, si el slot cabe dentro del horario del doctor y no se cruza con ninguna cita activa, el sistema lo considera disponible y lo incluye en la respuesta final.

Esto significa que la disponibilidad no es una suposición, sino el resultado de una comparación concreta entre horarios y ocupación real.

12.4. Ejemplo práctico paso a paso

Supongamos que un doctor tiene configurado horario el lunes de ocho de la mañana a doce del mediodía.

Ahora imaginemos que un cliente consulta la disponibilidad para ese doctor el lunes y pide slots de treinta minutos.

Lo primero que hace el sistema es verificar que el doctor exista.

Luego toma la fecha consultada, identifica que corresponde a un lunes y busca los horarios activos del doctor para ese día. Encuentra la franja de ocho a doce.

Después consulta las citas activas del doctor en esa fecha. Supongamos que ya existen dos citas: una de ocho y treinta a nueve y otra de diez a diez y treinta.

Con esa información, el sistema genera los posibles bloques de treinta minutos dentro de la franja de trabajo:

| | | | | |
|-------|---|---------|---|---------|
| ocho | a | ocho | y | treinta |
| ocho | y | treinta | a | nueve |
| nueve | a | nueve | y | treinta |
| nueve | y | treinta | a | diez |
| diez | a | diez | y | treinta |
| diez | y | treinta | a | once |
| once | a | once | y | treinta |

once y treinta a doce

Luego compara cada bloque con las citas existentes.

El slot de ocho a ocho y treinta no se cruza con ninguna cita, así que queda libre.

El slot de ocho y treinta a nueve coincide con una cita ya reservada, así que queda ocupado.

El slot de nueve a nueve y treinta queda libre.



El slot de nueve y treinta a diez queda libre.

El slot de diez a diez y treinta coincide con la segunda cita, así que queda ocupado.

Los demás slots que no se crucen con una cita activa quedan disponibles.

Finalmente, el sistema devuelve solo los slots libres. Esa lista es la disponibilidad real del doctor para esa fecha y esa duración.

13. Cómo funcionan los reportes

El módulo de reportes permite observar el comportamiento del sistema desde una perspectiva más analítica. Mientras otros módulos se enfocan en registrar o controlar operaciones, los reportes sirven para resumir información y convertirla en métricas útiles para la gestión.

Este punto es importante porque demuestra que el proyecto no solo organiza citas, sino que también permite analizar cómo está funcionando la operación de la clínica universitaria.

Los reportes se apoyan en consultas agregadas del repository y en una capa de servicio que transforma esos resultados en respuestas entendibles para la API.

13.1. Reporte de ocupación de consultorios

Este reporte responde a la pregunta de qué tan usados están los consultorios dentro de un rango de fechas.

Para cada consultorio, el sistema calcula cuántas citas tuvo en total, cuántas de esas citas se completaron, cuántas se cancelaron y cuál es su porcentaje de ocupación.

La idea del porcentaje de ocupación es mostrar qué tan aprovechado estuvo un consultorio en términos de atenciones efectivamente realizadas dentro del total de citas del periodo.

Este reporte es útil para evaluar uso de infraestructura, detectar espacios poco aprovechados y comparar comportamiento entre distintos consultorios.

13.2. Reporte de productividad de doctores

Este reporte permite observar el desempeño operativo de cada doctor dentro de un rango de fechas.

Para cada profesional, el sistema calcula cuántas citas tuvo en total, cuántas completó, cuántas se cancelaron, cuántas terminaron en no show y cuál es su tasa de completitud.

La tasa de completitud ayuda a resumir el porcentaje de citas que realmente terminaron como atención realizada frente al total registrado.

Este reporte es útil para analizar carga de trabajo, productividad y comportamiento de la agenda médica por especialista.



13.3. Reporte de pacientes con más no-show

Este reporte se enfoca en los pacientes que presentan mayores niveles de inasistencia.

Para cada paciente identificado dentro del rango consultado, el sistema devuelve información básica como nombre, correo, matrícula y la cantidad de citas marcadas como no show.

Su objetivo es permitir que la institución detecte patrones de incumplimiento, tome decisiones administrativas y haga seguimiento sobre comportamiento de asistencia.

Este reporte es especialmente útil porque convierte el estado no show en información práctica para la gestión.

13.4. Qué consultas del repository soportan estos reportes

Los reportes se sostienen sobre consultas agregadas especializadas dentro del repository de citas.

El reporte de ocupación de consultorios se apoya en una consulta que agrupa citas por consultorio y cuenta cuántas existen en el rango, cuántas se completaron y cuántas se cancelaron.

El reporte de productividad de doctores se apoya en una consulta que agrupa citas por doctor y resume totales, completadas, canceladas y no show.

El reporte de pacientes con más no show se apoya en una consulta que agrupa citas por paciente filtrando aquellas que quedaron en estado no show.

Estas consultas son importantes porque permiten transformar grandes volúmenes de registros individuales en resultados agregados mucho más útiles para el análisis.

13.5. Qué significan los datos agregados

Los datos agregados son resultados que no muestran cada cita por separado, sino un resumen calculado a partir de muchas citas.

Por ejemplo, en lugar de listar una por una todas las citas de un consultorio, el sistema devuelve un total de citas, un total de completadas, un total de canceladas y un porcentaje de ocupación.

Lo mismo ocurre con la productividad médica. En lugar de responder con cada cita del doctor, el sistema resume cuántas tuvo y cómo se distribuyeron según su resultado.

Estos datos agregados son importantes porque ayudan a ver patrones y no solo registros aislados. Permiten responder preguntas administrativas con rapidez y convierten la base de datos en una fuente de información útil para la toma de decisiones.

En otras palabras, el valor del módulo de reportes está en que transforma datos operativos en métricas comprensibles.



14. Qué se prueba y por qué se prueba

Las pruebas del proyecto sirven para demostrar que el sistema no solo fue construido, sino también validado. Esto es muy importante en la sustentación porque una API backend no se evalúa únicamente por el código que tiene, sino también por la evidencia de que sus reglas de negocio, sus consultas y sus endpoints realmente funcionan.

En este proyecto, las pruebas cumplen varios propósitos. Verifican que la lógica principal de las citas sea correcta. Comprueban que los conflictos de horario se detecten como corresponde. Validan que las transiciones de estado no permitan acciones inválidas. También aseguran que los endpoints devuelvan los códigos HTTP correctos y que las consultas complejas contra la base de datos produzcan resultados reales.

Gracias a esto, el sistema no depende solo de que el desarrollador diga que funciona. Existe evidencia automatizada que respalda que los comportamientos importantes fueron comprobados.

14.1. Qué son pruebas unitarias

Las pruebas unitarias son pruebas enfocadas en una sola pieza de lógica a la vez. Su objetivo es comprobar que una clase o método se comporte correctamente de forma aislada, sin depender de toda la aplicación completa.

En este proyecto, las pruebas unitarias se usan principalmente sobre la capa service. Esto tiene mucho sentido porque la lógica de negocio está allí. Por ejemplo, se prueban las validaciones de creación de una cita, las transiciones de estado, la prevención de duplicados en módulos como pacientes o doctores y la lógica de disponibilidad.

En una prueba unitaria, las dependencias externas no se usan de forma real. En lugar de eso, se simulan. Eso permite concentrarse en el comportamiento del método que se quiere probar. Por ejemplo, cuando se prueba una cita, no hace falta consultar una base de datos real. Basta con simular que el repositorio devuelve un paciente activo o que existe un conflicto de horario.

El valor de este tipo de prueba está en que permite revisar la lógica de negocio de forma rápida y controlada. Si una regla falla, la prueba lo detecta sin necesidad de levantar todo el sistema.

14.2. Qué son pruebas de integración

Las pruebas de integración verifican que varias piezas del sistema funcionen bien juntas. En este proyecto, se usan especialmente para validar el acceso real a la base de datos y el comportamiento de las consultas complejas.

A diferencia de una prueba unitaria, una prueba de integración no se queda solo en la lógica aislada. Aquí sí se pone a trabajar la aplicación con una base de datos real de prueba. Eso permite comprobar que las entidades están bien mapeadas, que los repositorios ejecutan correctamente sus consultas y que los flujos importantes funcionan de principio a fin.



En este proyecto, las pruebas de integración son muy valiosas porque el sistema depende de consultas reales para detectar traslapes y construir reportes. Por eso no basta con simular todo. También hace falta probar que esas consultas funcionen sobre una base relacional real.

En términos sencillos, las pruebas de integración responden a esta pregunta

Cuando el sistema se conecta de verdad con la base de datos todo sigue funcionando como se espera

14.3. Qué son pruebas de controller

Las pruebas de controller verifican la capa de entrada de la API. Se enfocan en comprobar que los endpoints respondan correctamente frente a distintas solicitudes.

Estas pruebas no analizan toda la lógica interna en profundidad. Su objetivo principal es comprobar que una ruta existe, que responde con el código HTTP correcto, que maneja errores adecuadamente y que devuelve información en una estructura razonable.

Por ejemplo, en este proyecto se prueban escenarios como creación correcta de una cita, errores de validación, recursos inexistentes, conflictos de horario y respuestas exitosas de algunos reportes.

Este tipo de prueba es importante porque demuestra que la API no solo tiene reglas internas correctas, sino también una interfaz HTTP bien construida hacia el exterior.

14.4. Qué archivos de test existen en este proyecto

El proyecto tiene un conjunto bastante amplio de archivos de prueba distribuidos por tipo.

En pruebas unitarias de services destacan archivos como AppointmentServiceImplTest, AvailabilityServiceImplTest, DoctorScheduleServiceImplTest, DoctorServiceImplTest, PatientServiceImplTest, SpecialtyServiceImplTest, AppointmentTypeServiceImplTest, OfficeServiceImplTest y ReportServiceImplTest.

En pruebas de integración destacan AppointmentRepositoryIntegrationTest, DoctorRepositoryIntegrationTest, DoctorScheduleRepositoryIntegrationTest, OfficeRepositoryIntegrationTest y PatientRepositoryIntegrationTest.

En pruebas de controller destacan AppointmentControllerTest, AvailabilityControllerTest, DoctorControllerTest, PatientControllerTest y ReportControllerTest.

También existe ClinicReservationsApplicationTest, que valida el arranque general del contexto de la aplicación.

En conjunto, esto muestra que el sistema fue probado desde varios ángulos y no solo desde una única perspectiva.



14.5. Qué protege cada grupo de pruebas

Las pruebas unitarias protegen la lógica de negocio. Gracias a ellas, el equipo puede asegurar que crear una cita, cancelarla, completarla o calcular disponibilidad funcione según las reglas definidas.

Las pruebas de integración protegen la relación entre el código y la base de datos. Verifican que las consultas reales funcionen correctamente, que los datos se persistan bien y que los reportes se construyan sobre resultados válidos.

Las pruebas de controller protegen la interfaz HTTP del proyecto. Aseguran que los endpoints respondan correctamente, que usen los códigos de estado adecuados y que el sistema maneje bien los errores visibles para el cliente.

La prueba de contexto protege el arranque general de la aplicación. Sirve para detectar problemas de configuración que podrían impedir que el sistema levante correctamente.

Dicho de forma simple, cada grupo de pruebas protege una capa distinta del sistema. Juntas forman una evidencia fuerte de calidad técnica.

14.6. Qué significa que mvn clean test pase con todos los tests

Que mvn clean test pase con todos los tests significa que la aplicación pudo compilar, levantar su entorno de pruebas y ejecutar exitosamente toda la suite automatizada sin fallos.

Eso es importante porque demuestra varias cosas al mismo tiempo.

Demuestra que la lógica de negocio principal funciona según lo esperado.

Demuestra que los endpoints más importantes responden bien.

Demuestra que las consultas sobre la base de datos real son válidas.

Demuestra que el contexto general de la aplicación puede arrancar sin errores graves.

En la práctica, esto le da mucha solidez al proyecto frente a la sustentación, porque ya no se trata solo de una explicación verbal del sistema, sino de una prueba real de que su comportamiento fue validado automáticamente.

15. Archivos que sí o sí deben conocerse para defender el proyecto

Una de las mayores dificultades en una sustentación grupal es que no todos los integrantes tienen el mismo dominio del proyecto. Por eso es clave priorizar. Nadie necesita memorizar cada línea del sistema, pero sí debe ubicar los archivos más importantes y saber qué papel cumplen.



Esta sección sirve como mapa de estudio. La idea es que cualquier compañero sepa cuáles son los archivos verdaderamente críticos para entender el proyecto y cuáles pueden revisarse de manera más superficial.

15.1. Top 10 archivos más importantes

1. AppointmentServiceImpl.java

Este es el archivo más importante del proyecto. Aquí vive la lógica principal de creación de citas, validaciones de negocio y transiciones de estado. Si alguien entiende este archivo, entiende una gran parte de la complejidad del sistema.

2. AppointmentRepository.java

Es el repositorio más fuerte del proyecto. Contiene las consultas necesarias para detectar traslapes, filtrar citas, calcular disponibilidad y soportar los reportes. Es clave para explicar cómo el sistema usa la base de datos de forma inteligente.

3. Appointment.java

Es la entidad central del sistema. Conecta paciente, doctor, consultorio y tipo de cita. También guarda inicio, fin, estado y notas. Es el mejor archivo para explicar por qué la cita médica es el corazón del proyecto.

4. AppointmentController.java

Expone la mayor parte de la funcionalidad crítica del backend. Sirve para mostrar cómo se crean citas, cómo se filtran y cómo se mueven entre estados.

5. AvailabilityServiceImpl.java

Contiene el algoritmo de cálculo de slots libres. Es muy útil para defender la parte más dinámica y menos trivial del sistema.

6. ReportServiceImpl.java

Explica cómo el sistema transforma consultas agregadas en reportes útiles. Es el archivo más importante para sostener la parte analítica del proyecto.

7. GlobalExceptionHandler.java

Permite explicar cómo el sistema maneja errores y por qué la API responde con códigos y mensajes adecuados.

8. AppointmentStatus.java

Aunque es pequeño, es muy importante para entender la máquina de estados de la cita y el ciclo de vida del proceso.

9. AppointmentServiceImplTest.java

Es uno de los mejores respaldos del proyecto porque demuestra con pruebas la lógica central de citas y transiciones de estado.



10. AppointmentRepositoryIntegrationTest.java

Este archivo es importante porque respalda que las consultas y flujos principales funcionan realmente con base de datos real y no solo con simulaciones.

15.2. Qué responder si preguntan por cada uno

Si preguntan por AppointmentServiceImpl.java se debe responder que allí está la lógica de negocio principal del proyecto, especialmente la creación de citas, la validación de conflictos y las transiciones de estado.

Si preguntan por AppointmentRepository.java se debe responder que allí están las consultas más importantes del sistema, incluyendo detección de traslapes, filtros y soporte de reportes.

Si preguntan por Appointment.java se debe responder que representa la entidad central del sistema y que conecta paciente, doctor, consultorio y tipo de cita en una misma cita médica.

Si preguntan por AppointmentController.java se debe responder que expone los endpoints principales del proyecto relacionados con citas y que actúa como punto de entrada HTTP para esa funcionalidad.

Si preguntan por AvailabilityServiceImpl.java se debe responder que allí se calcula la disponibilidad real del doctor a partir de sus horarios y de las citas activas existentes.

Si preguntan por ReportServiceImpl.java se debe responder que toma resultados agregados del repository y los convierte en respuestas claras para reportes de ocupación, productividad e inasistencias.

Si preguntan por GlobalExceptionHandler.java se debe responder que centraliza el manejo de errores y permite devolver respuestas HTTP ordenadas y comprensibles.

Si preguntan por AppointmentStatus.java se debe responder que define los estados posibles de una cita y sirve como base para la máquina de estados del sistema.

Si preguntan por AppointmentServiceImplTest.java se debe responder que prueba la lógica principal de negocio de las citas y demuestra que las reglas importantes fueron verificadas.

Si preguntan por AppointmentRepositoryIntegrationTest.java se debe responder que valida flujos completos y consultas reales del sistema usando una base de datos de prueba.

15.3. Qué no vale la pena memorizar en detalle

No vale la pena memorizar cada DTO individual ni cada mapper pequeño de forma exhaustiva. Es suficiente con entender para qué existen y qué función general cumplen.

Tampoco vale la pena aprender de memoria todos los servicios CRUD simples si no aportan lógica nueva. Por ejemplo, varios servicios de catálogos siguen patrones parecidos de crear, buscar y actualizar.



No es necesario memorizar todas las anotaciones de cada clase ni cada detalle del archivo de configuración si no cambia el comportamiento esencial del sistema.

Tampoco hace falta recitar todos los endpoints menores uno por uno. Es más útil dominar los endpoints principales del módulo de citas, disponibilidad y reportes, y luego saber que los demás módulos exponen operaciones CRUD básicas.

En general, lo importante es priorizar aquello que ayuda a explicar el valor real del proyecto y no perder tiempo en detalles secundarios que no suelen ser el centro de la sustentación

16. Preguntas probables de sustentación y cómo responderlas

La sustentación no solo consiste en mostrar el proyecto, sino en responder preguntas de forma clara y coherente. Por eso conviene anticipar cuáles son las preguntas más probables y tener una forma sencilla de responderlas.

La idea de esta sección no es que el grupo memorice respuestas exactas como robot, sino que entienda qué tipo de preguntas pueden hacerle y qué elementos del proyecto sirven para responderlas con seguridad.

16.1. Preguntas de arquitectura

Pregunta probable

Qué arquitectura usa el proyecto

Respuesta sugerida

El proyecto usa una arquitectura por capas. La capa controller recibe las solicitudes HTTP. La capa service aplica la lógica de negocio. La capa repository consulta y guarda información en la base de datos. Las entidades modelan el dominio y los DTOs y mappers controlan cómo entra y sale la información de la API.

Pregunta probable

Por qué separaron interface e implementación en los services

Respuesta sugerida

Porque eso permite una arquitectura más limpia y desacoplada. El controller depende del contrato general del servicio y no de una implementación concreta. Además, facilita pruebas y mantenimiento.

Pregunta probable

Por qué no dejaron toda la lógica dentro del controller

Respuesta sugerida



Porque el controller debe encargarse de recibir solicitudes y delegar. Si toda la lógica quedara allí, el código se volvería difícil de mantener, probar y explicar. La lógica real debe vivir en la capa service.

16.2. Preguntas de reglas de negocio

Pregunta probable

Qué valida el sistema al crear una cita

Respuesta sugerida

Valida que existan paciente, doctor, consultorio y tipo de cita. Valida que paciente y doctor estén activos y que el consultorio esté operativo. Valida que la cita no esté en el pasado, que caiga dentro del horario del doctor, que se calcule correctamente la hora final y que no haya traslapes con otras citas del doctor, del paciente o del consultorio.

Pregunta probable

Cómo evitan choques de agenda

Respuesta sugerida

El sistema ejecuta validaciones de traslape en la capa de negocio apoyadas en consultas del repository. Revisa por separado conflictos del doctor, del consultorio y del paciente.

Pregunta probable

Qué estados puede tener una cita

Respuesta sugerida

Una cita puede estar agendada confirmada cancelada completada o marcada como no asistida. El sistema controla que solo ciertas transiciones sean válidas.

16.3. Preguntas de base de datos

Pregunta probable

Cuántas entidades tiene el sistema y cuál es la más importante

Respuesta sugerida

El sistema tiene siete entidades principales. La más importante es la cita médica porque conecta paciente doctor consultorio y tipo de cita y concentra la lógica principal del proyecto.

Pregunta probable

Cómo se relacionan las entidades

Respuesta sugerida

La especialidad agrupa doctores. Un doctor tiene varios horarios. Paciente doctor consultorio y tipo de cita se relacionan todos con la cita médica. La cita es el punto donde convergen las demás entidades.



Pregunta probable

Por qué usaron una base de datos relacional

Respuesta sugerida

Porque el problema depende de relaciones claras entre varias entidades y de restricciones de integridad. Una cita necesita conectar varias piezas del negocio con consistencia.

16.4. Preguntas de reportes

Pregunta probable

Qué reportes genera el sistema

Respuesta sugerida

Genera reporte de ocupación de consultorios, reporte de productividad de doctores y reporte de pacientes con mayor cantidad de no show.

Pregunta probable

Cómo construyen los reportes

Respuesta sugerida

Se apoyan en consultas agregadas del repository que agrupan información por consultorio, doctor o paciente. Luego la capa service transforma esos resultados en respuestas claras para la API.

Pregunta probable

Por qué los reportes son importantes

Respuesta sugerida

Porque convierten el sistema en una herramienta no solo operativa sino también analítica. Permiten medir uso de consultorios, productividad médica y comportamiento de inasistencia.

16.5. Preguntas de testing

Pregunta probable

Qué tipos de pruebas tiene el proyecto

Respuesta sugerida

Tiene pruebas unitarias, pruebas de integración, pruebas de controller y una prueba de contexto general.

Pregunta probable

Qué diferencia hay entre esas pruebas

Respuesta sugerida



Las unitarias validan lógica aislada. Las de integración validan comportamiento real con base de datos. Las de controller validan la capa HTTP. La de contexto verifica que la aplicación arranque correctamente.

Pregunta probable

Por qué es importante que mvn clean test pase

Respuesta sugerida

Porque demuestra que la lógica, los endpoints, la integración con la base de datos y el arranque general del sistema fueron verificados automáticamente sin errores.

16.6. Preguntas sobre decisiones técnicas

Pregunta probable

Por qué usaron Java y Spring Boot

Respuesta sugerida

Porque era parte del stack exigido por el taller y porque es un ecosistema muy adecuado para construir APIs backend robustas con buena organización por capas.

Pregunta probable

Por qué usaron PostgreSQL

Respuesta sugerida

Porque es una base de datos relacional sólida y encaja bien con la necesidad de manejar relaciones fuertes entre entidades y consultas agregadas.

Pregunta probable

Por qué usan DTOs y mappers

Respuesta sugerida

Para separar el modelo interno de la base de datos de lo que la API recibe y devuelve al cliente. Eso mejora seguridad, claridad y organización del proyecto.

17. Errores comunes o confusiones que deben evitar los compañeros

En una sustentación grupal no solo importa saber cosas, sino también evitar errores que hagan ver al grupo desordenado o confundido. Muchas veces una respuesta equivocada no nace de ignorancia total, sino de mezclar conceptos parecidos.

Por eso conviene tener muy claras algunas confusiones típicas que deben evitarse.



17.1. No confundir entidad con DTO

Una entidad representa el modelo interno y normalmente está ligada a la base de datos. Un DTO es un objeto de transferencia usado para entrada o salida de la API. No son lo mismo y no cumplen la misma función.

Decir que una entidad y un DTO son iguales sería mostrar que no se entendió la arquitectura del sistema.

17.2. No confundir repository con service

El repository se encarga de consultar y guardar datos. El service se encarga de la lógica de negocio.

Si alguien dice que el repository valida toda la lógica o que el service solo guarda información, está invirtiendo mal los roles de las capas.

17.3. No decir que el controller “hace toda la lógica”

El controller es solo la puerta de entrada de la API. Su trabajo es recibir la solicitud y delegar. La lógica importante vive en la capa service.

Decir que el controller hace todo el trabajo sería una mala explicación de la arquitectura.

17.4. No olvidar que endAt lo calcula el servidor

La hora final de una cita no la envía el cliente. El sistema la calcula automáticamente a partir de la hora inicial y la duración del tipo de cita.

Olvidar esto es perder una de las reglas más características del proyecto.

17.5. No olvidar los 3 tipos de reportes principales

Los reportes principales del proyecto son ocupación de consultorios, productividad de doctores y pacientes con más no show.

Si un compañero no recuerda estos tres, está perdiendo una parte importante de la funcionalidad analítica del sistema.

17.6. No olvidar las transiciones válidas de estado

Una cita no puede cambiar libremente de estado. Nace agendada, puede confirmarse y luego completarse cancelarse o quedar en no show según las reglas.

Olvidar esto puede llevar a respuestas incoherentes en preguntas sobre el flujo principal del sistema.



18. Resumen final del proyecto en una página

Esta sección funciona como una hoja de repaso rápido para antes de entrar a la sustentación. La idea es condensar lo más importante del sistema en una versión corta pero clara.

18.1. Qué problema resuelve

El proyecto resuelve la mala gestión de citas médicas dentro de una clínica universitaria. Evita choques de horario, duplicidad de registros, mala utilización de consultorios, citas fuera del horario del doctor y falta de trazabilidad en la operación.

18.2. Cómo está construido

Está construido como una API REST backend desarrollada con Java y Spring Boot. Usa una arquitectura por capas en la que los controllers reciben las solicitudes, los services aplican la lógica de negocio, los repositories acceden a la base de datos y las entidades modelan el dominio. Además, utiliza DTOs, mappers y manejo global de excepciones.

18.3. Qué reglas importantes cumple

El sistema valida la existencia y el estado de paciente doctor consultorio y tipo de cita. No permite citas en el pasado. Calcula automáticamente la hora final de cada cita. Obliga a que las reservas estén dentro del horario real del doctor. Evita traslapes de doctor consultorio y paciente. Controla una máquina de estados para las citas y permite cancelación con motivo, completado con observaciones y registro de no show.

18.4. Qué pruebas lo respaldan

El proyecto cuenta con pruebas unitarias para la lógica de negocio, pruebas de integración para validar el comportamiento real con base de datos y pruebas de controller para comprobar la capa HTTP. También incluye una prueba de contexto general. Esto demuestra que el sistema fue validado más allá de la simple compilación.

18.5. Qué lo hace defendible frente a la rúbrica

Lo hace defendible el hecho de que cumple con un modelo de datos claro, repositories con consultas útiles, services con reglas de negocio reales, controllers que exponen la funcionalidad principal, pruebas automatizadas en varias capas y documentación suficiente para explicar la solución. Además, su módulo de citas, disponibilidad y reportes demuestra una complejidad real acorde con el objetivo del taller.

En una frase final, este proyecto puede resumirse así

Es un backend de gestión de citas médicas universitarias que no solo registra información, sino que la organiza, la valida, la protege frente a conflictos y la convierte en resultados útiles para la operación y la toma de decisiones.



Informe técnico — Proyecto Web

Versión: 1